

AI VIET NAM – AI COURSE 2025

Tutorial: Dịch thuật Hoa-Việt (Olympic AI)

Nguyễn Quốc Thái
Xỉn Quý Hùng

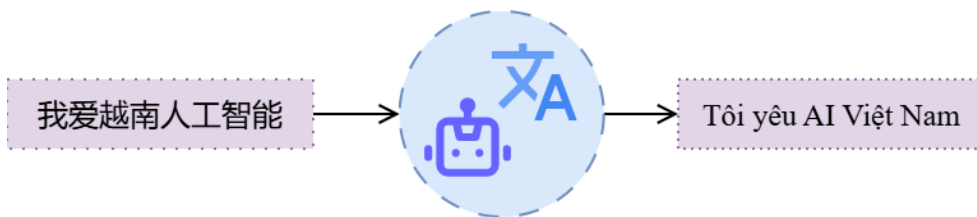
Hồ Quang Hiến
Đinh Quang Vinh

I. Giới thiệu

Cuộc thi Olympic AI 2025 (OlpAI'25) được Hội Tin học Việt Nam tổ chức lần đầu tiên, dành cho các bạn học sinh sinh viên có đam mê và muốn thử sức với các bài toán trong lĩnh vực Trí tuệ nhân tạo (AI). Tuy mới tổ chức lần đầu, cuộc thi vẫn nhận được nhiều sự quan tâm và tham gia từ các thí sinh khắp cả nước. OlpAI'25 mang đến nhiều đề thi thú vị, nằm trong nhiều lĩnh vực: Học máy (ML), Thị giác máy tính (CV) và Xử lý ngôn ngữ tự nhiên (NLP).

Cuộc thi gồm 2 vòng: Vòng khu vực (theo các miền Bắc - Trung - Nam) và vòng chung kết. Ở vòng đầu tiên, mỗi khu vực sẽ do một trường trong khu vực đó đăng cai. Trong đó, vòng thi miền Bắc sẽ do Học viện Công nghệ Bưu chính Viễn thông (PTIT) đăng cai. Đề thi ở mỗi miền sẽ khác nhau, do đơn vị đăng cai tổ chức.

Ở vòng miền Bắc, các thí sinh cần phải giải quyết 2 bài toán thuộc 2 lĩnh vực khác nhau: CV và NLP. Đối với NLP, chủ đề được đưa ra là **Dịch thuật Hoa - Việt**, một bài toán thuộc nhánh Machine Translation (MT).

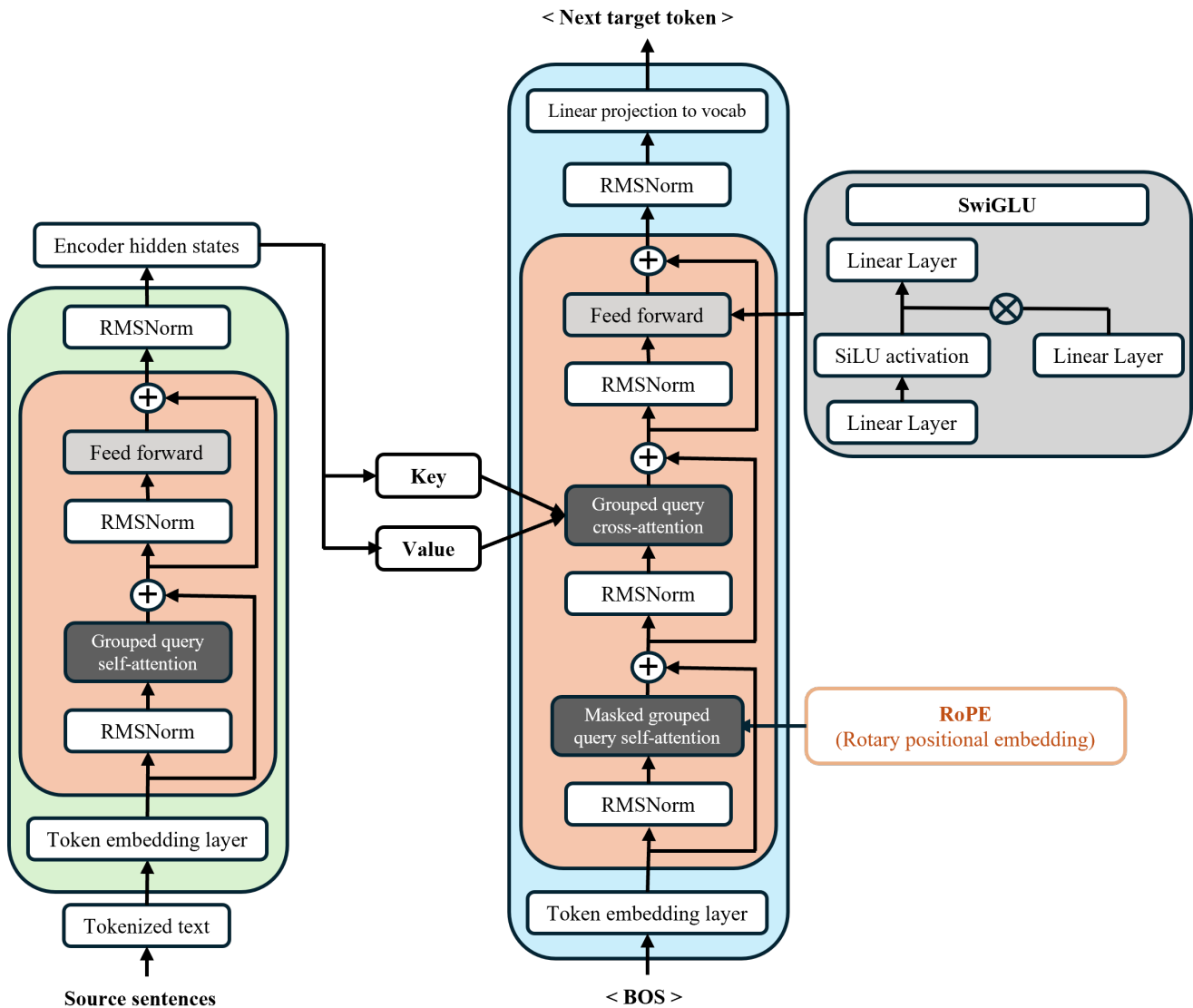


Hình 1: Minh họa đề thi dịch thuật Hoa - Việt.

Đề thi nhấn mạnh tính thực tế của các mô hình dịch thuật Hoa - Việt trước bối cảnh toàn cầu hóa, khi việc giao lưu về kinh tế, văn hóa cũng như học thuật giữa Việt Nam và Trung Quốc ngày càng được đẩy mạnh. Lúc này, các mô hình có khả năng tự động dịch giữa hai ngôn ngữ tiếng Việt và tiếng Trung Quốc càng trở nên hữu ích, có vai trò là công cụ giúp phá vỡ các rào cản về ngôn ngữ. Nhờ vào học sâu (Deep learning), các mô hình này càng trở nên mạnh mẽ, có

thể kể đến các hệ thống dịch thuật lớn như Google Translate, hoặc các mô hình ngôn ngữ lớn (LLMs) đang ngày càng được sử dụng rộng rãi trong tác vụ dịch thuật hằng ngày.

Bài toán MT là một trong những bài kinh điển của lĩnh vực NLP. Ở bài toán này, chúng ta sẽ xây dựng một mô hình có khả năng tự động dịch từ ngôn ngữ này sang ngôn ngữ khác. Cụ thể, với đề bài, các thí sinh sẽ phải huấn luyện một mô hình có khả năng dịch từ Tiếng Trung (Giản thể) sang tiếng Việt.



Hình 2: Mô hình Transformer Encoder-Decoder được xây dựng cho bài toán dịch thuật.

Mục lục

I.	Giới thiệu	1
II.	Phân tích đề thi	4
II.1.	Thể lệ và yêu cầu của cuộc thi	4
II.2.	Hướng triển khai	5
II.3.	Dữ liệu	5
II.4.	Kiến thức cần có	6
III.	Giới thiệu baseline	8
IV.	Bổ sung kiến thức về Transformer	18
IV.1.	RMSNorm	18
IV.2.	RoPE	19
IV.3.	Grouped-query attention	20
IV.4.	SwiGLU	21
IV.5.	Kiến trúc mô hình Transformer Encoder-Decoder được sử dụng	22
V.	Mô hình Transformer Encoder-Decoder cho bài toán dịch thuật Hoa-Việt	24
VI.	Hướng phát triển và cải tiến mô hình	41
VI.1.	Thử nghiệm đa cấu hình: kiến trúc, từ vựng và lịch huấn luyện	41
VI.2.	Khai thác Back-Translation để mở rộng dữ liệu	41
VI.3.	Regularization nâng cao: R-Drop và tối ưu hoá SAM	42
VI.4.	Mở rộng khung contrastive với cặp dương từ nhiều phiên bản của cùng một câu	43
VI.5.	Khảo sát các chiến lược giải mã: beam search, top- k , top- p	43
VI.6.	Vòng lặp suy luận, phân tích lỗi và cải thiện mô hình	44
	Phụ lục	46

II. Phân tích đề thi

II.1. Thể lệ và yêu cầu của cuộc thi

Đến với cuộc thi, ban tổ chức sẽ cung cấp các tài nguyên bài thi gồm: Dữ liệu train, dữ liệu test và code baseline.

Dựa vào dữ liệu train, nhiệm vụ của thí sinh là **huấn luyện mô hình dịch Hoa - Việt**, với khả năng nhận đầu vào là một câu tiếng Trung và dịch ra câu tiếng Việt tương ứng.

Về dữ liệu test, ban tổ chức cung cấp 2 loại: **Public test** được nhận vào đầu buổi thi để đánh giá kết quả và **Private test** được mở vào cuối buổi thi.

Ngoài ra, các thí sinh cũng sẽ được cung cấp một file code *baseline*, bao gồm đầy đủ các bước để xây dựng mô hình. Các thí sinh có thể dựa vào baseline, phân tích các ưu khuyết điểm để làm cơ sở phát triển cho mô hình của mình.

Kết quả sẽ được đánh giá bằng **SacreBLEU**. Đây là một độ đo được sử dụng phổ biến đối với nhiều bài toán trong NLP, trong đó có MT. SacreBLEU được tính bằng:

$$\text{SacreBLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

Trong đó:

- p_n : Độ chính xác n-gram bậc n , tính bằng

$$p_n = \frac{\sum_{\text{ngram} \in \text{hyp}_n} \min(\text{count}_{\text{hyp}}(\text{ngram}), \max_{\text{ref}} \text{count}_{\text{ref}}(\text{ngram}))}{\sum_{\text{ngram} \in \text{hyp}_n} \text{count}_{\text{hyp}}(\text{ngram})}$$

- w_n : Trọng số, thường là $\frac{1}{4}$ cho $n = 1, 2, 3, 4$.
- BP : Hệ số phạt độ ngắn, tính bằng

$$BP = \begin{cases} 1, & \text{nếu } c > r \\ \exp(1 - \frac{r}{c}), & \text{nếu } c \leq r \end{cases}$$

- $\text{count}_{\text{hyp}}(\text{ngram})$: Số lần xuất hiện của n-gram trong bản dịch máy.
- $\text{count}_{\text{ref}}(\text{ngram})$: Số lần xuất hiện của n-gram trong bản dịch tham chiếu.
- c : Tổng độ dài của bản dịch hệ thống.
- r : Tổng độ dài của bản dịch tham chiếu.

Ngoài ra, điểm SacreBLEU còn được chuẩn hóa (Normalize) theo công thức của cuộc thi, nhưng nhìn chung chúng ta vẫn chỉ quan tâm đến việc đạt SacreBLEU càng cao càng tốt.

Bên cạnh đề thi cơ bản là xây dựng mô hình dịch từ Tiếng Trung sang tiếng Việt, cuộc thi vẫn có một số yêu cầu cơ bản sau:

- Các thí sinh **không được sử dụng các mô hình huấn luyện sẵn (pre-trained)** trong bài làm của mình. Điều này có nghĩa là thí sinh không được sử dụng các mô hình huấn luyện sẵn (pre-trained) như **BERT, BART**, ... cho bất kỳ bước xử lý nào trong bài làm của mình. .
- Các thí sinh **không được sử dụng dữ liệu từ bên ngoài** bổ sung cho tập train đã được cung cấp. Điều này có nghĩa là chiến thuật tăng cường dữ liệu bằng cách bổ sung data từ bên ngoài cuộc thi sẽ không được chấp nhận. Tuy nhiên, các thí sinh vẫn **được tạo dữ liệu nhân tạo (synthetic)** dựa trên dữ liệu của ban tổ chức.

II.2. Hướng triển khai

Đối với bài toán MT, chúng ta có một số cách tiếp cận như:

- **Rule-based**: Dịch thuật theo tập luật được định sẵn từ trước
- **Statistical Machine Translation (SMT)**: Dựa trên xác suất xuất hiện của từ/cụm từ.
- **Neural Machine Translation (NMT)**: Huấn luyện mô hình học sâu

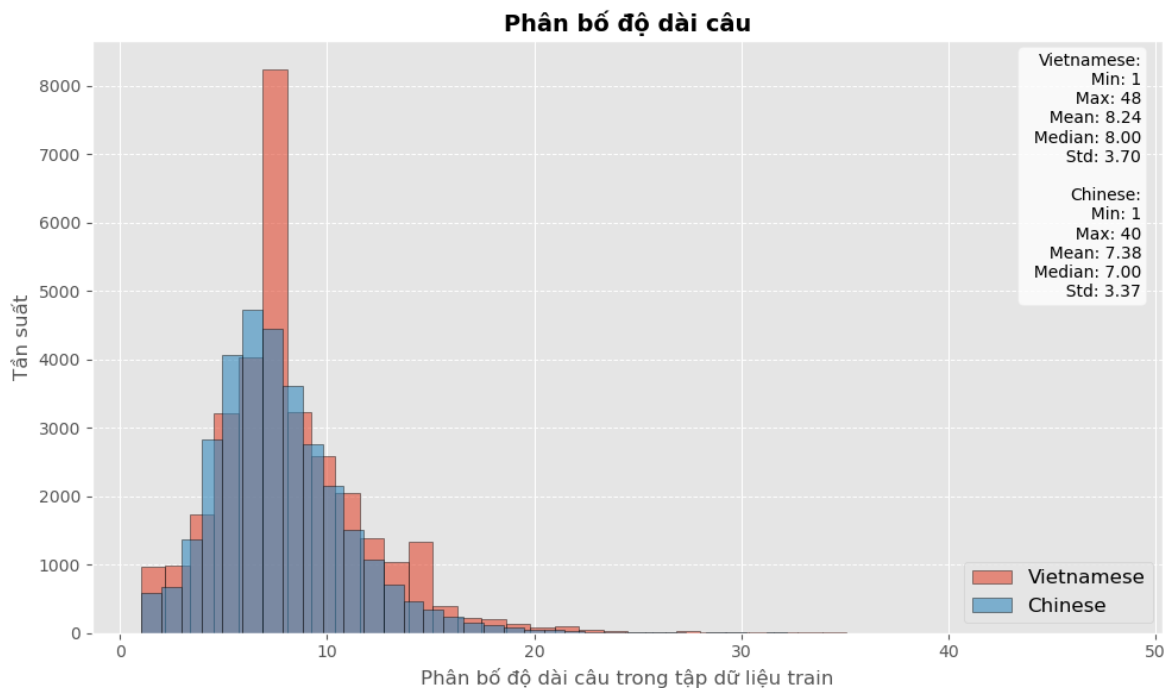
Có thể thấy rằng, cách làm phổ biến cũng như được nhiều thí sinh lựa chọn vẫn là **NMT**. Sự thành công của các mô hình học sâu chính là minh chứng cho sức mạnh của **NMT**, với kết quả tốt hơn nhiều so với hai phương pháp còn lại. Bên cạnh đó, code baseline cũng được triển khai theo hướng NMT. Do đó, để được kết quả cao, chúng ta vẫn nên lựa chọn NMT.

II.3. Dữ liệu

Bộ dữ liệu được cung cấp được chia làm ba phần:

- **Tập train**: Bao gồm 2 file là `train.zh` và `train.vi`. Trong đó, `train.zh` bao gồm các văn bản tiếng Trung, còn `train.vi` bao gồm các văn bản tiếng Việt.
- **Tập public test**: `public_test.zh`
- **Tập private test**: `private_test.zh`

Trong đó, tập huấn luyện bao gồm 32,061 samples. Đối với bài toán MT, đây là số lượng không quá nhỏ nhưng cũng không quá lớn. Các sample khác nhau có độ dài không đều: có những câu rất ngắn (1-2 từ) và những câu rất dài (hơn 40 từ).



Hình 3: Tổng quan về độ dài câu trong tập huấn luyện.

II.4. Kiến thức cần có

Đối với cuộc thi này, chúng ta cần nắm bắt các kiến thức nền tảng của NLP cũng như bài toán MT, cả về lý thuyết và thực hành.

Về mặt lý thuyết, việc trang bị những kiến thức nền tảng cho bài toán MT sẽ là lợi thế lớn. Những kiến thức này bao gồm:

- **xử lý dữ liệu:** Nắm rõ các kỹ thuật cơ bản của NLP như data normalization hay tokenization (Word-based tokenization, Byte-pair-encoding (BPE),...). Ngoài ra, các kỹ thuật Data augmentation trong bài toán MT cũng quan trọng, giúp mô hình cải thiện khả năng tổng quát và tránh overfit.
- **Xây dựng các mô hình học sâu:** Hiểu một số mô hình cơ bản (RNN, GRU, Transformer). Các mô hình với kiến trúc mạnh mẽ với khả năng học tốt có thể cải thiện kết quả rất nhiều.
- **Kỹ thuật huấn luyện mô hình:** Lựa chọn hàm loss phù hợp (Cross-Entropy, Label Smoothing), sử dụng các optimizer hiện đại (Adam, AdamW, Adagrad, ...), áp dụng learning rate scheduler (warm-up, cosine decay), regularization (dropout, weight decay),...
- **Kỹ thuật Decode:** Biết một số kỹ thuật decode như Greedy decode, Beam search,...

Việc có những kiến thức sâu rộng trong bài toán MT là một lợi thế rất lớn, giúp cạnh tranh với các thí sinh khác. Điều đó đòi hỏi mỗi thí sinh cần liên tục cập nhật các kiến thức mới, tìm hiểu những kỹ thuật hiện đại và mạnh mẽ trong những năm gần đây.

Về mặt thực hành, chúng ta cần có một số nền tảng lập trình nhất định (biết cách sử dụng Python,...) và cần nắm rõ cách sử dụng các thư viện Deep learning như Pytorch, Tensorflow hay Keras.

III. Giới thiệu baseline

Ở phần này, chúng ta sẽ tìm hiểu và phân tích baseline do ban tổ chức cung cấp. Việc tìm hiểu baseline là vô cùng quan trọng, giúp tạo tiền đề cho những quyết định tiếp theo.

Trước hết, dữ liệu được tokenize bằng phương pháp **BPE**. Tiếp theo, một mô hình Seq2Seq với **Encoder** và **Decoder** đều sử dụng các khối GRU được xây dựng và tiến hành huấn luyện. Sau khi đã huấn luyện xong mô hình, baseline decode bằng phương pháp **Greedy decoding** lên tập dữ liệu test.

Bước 1: Configuration and Hyperparameters

```
1 # Data paths
2 TRAIN_SRC = "dataset/train/train.zh"
3 TRAIN_TGT = "dataset/train/train.vi"
4 TEST_SRC = "dataset/public_test/public_test.zh"
5 SAVE_DIR = "./checkpoints"
6 SPM_ZH_PREFIX = os.path.join(SAVE_DIR, "spm_zh")
7 SPM_VI_PREFIX = os.path.join(SAVE_DIR, "spm_vi")
8
9 # Model hyperparameters
10 VOCAB_SIZE = 3000
11 EMB_SIZE = 64
12 HID_SIZE = 128
13 BATCH_SIZE = 64
14 EPOCHS = 10
15 LR = 0.01
16 MAX_LEN = 80
17 SEED = 42
18
19 # Device configuration
20 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
21 print(f"Using device: {DEVICE}")
22
23 # Set random seeds for reproducibility
24 os.makedirs(SAVE_DIR, exist_ok=True)
25 random.seed(SEED)
26 torch.manual_seed(SEED)
27 if torch.cuda.is_available():
28     torch.cuda.manual_seed(SEED)
```

Giải thích: Ở phần này, baseline gán giá trị cho các hyperparameters chính trong bài. Trong đây, các thông số đáng chú ý là: `VOCAB_SIZE = 3000`, `EMB_SIZE = 64` và `HID_SIZE = 128`.

Nhận xét: Nhìn chung, các thông số này tương đối nhỏ. Một chiến thuật có thể thử nhằm cải thiện baseline chính là tăng giá trị của các hyperparameters kể trên, chẳng hạn như tăng giá trị của `VOCAB_SIZE` để tránh vấn đề Out-of-Vocabulary (OOV), hoặc tăng `EMB_SIZE` và `HID_SIZE` để nâng kích thước mô hình.

Bước 2: Data processing

```

1 def train_spm(input_file, model_prefix, vocab_size=VOCAB_SIZE):
2     """Train a SentencePiece BPE model."""
3     args = (
4         f"--input={input_file} --model_prefix={model_prefix} --vocab_size={vocab_size}
5         "
6         "--model_type=bpe --character_coverage=1.0 "
7         "--pad_id=0 --unk_id=1 --bos_id=2 --eos_id=3"
8     )
9     spm.SentencePieceTrainer.Train(args)
10    print(f"Trained SentencePiece model: {model_prefix}.model")
11
12 def load_sp(model_path):
13     """Load a trained SentencePiece model."""
14     sp = spm.SentencePieceProcessor()
15     sp.Load(model_path)
16     return sp
17
18 # Train Chinese tokenizer
19 tmp_zh = os.path.join(SAVE_DIR, "tmp_zh.txt")
20 if not os.path.exists(SPM_ZH_PREFIX + ".model"):
21     with open(tmp_zh, "w", encoding="utf-8") as f:
22         for s in train_src:
23             f.write(s + "\n")
24     train_spm(tmp_zh, SPM_ZH_PREFIX)
25
26 # Train Vietnamese tokenizer
27 tmp_vi = os.path.join(SAVE_DIR, "tmp_vi.txt")
28 if not os.path.exists(SPM_VI_PREFIX + ".model"):
29     with open(tmp_vi, "w", encoding="utf-8") as f:
30         for s in train_tgt:
31             f.write(s + "\n")
32     train_spm(tmp_vi, SPM_VI_PREFIX)
33
34 # Load tokenizers
35 sp_zh = load_sp(SPM_ZH_PREFIX + ".model")
36 sp_vi = load_sp(SPM_VI_PREFIX + ".model")

```

Giải thích: Ở phần này, baseline sử dụng thư viện **Sentencepiece** cho quá trình tokenize. Dựa vào biến *args* trong hàm *train_spm*, ta có thể dễ dàng nhận thấy những điều sau:

- **model_type=bpe:** Phương pháp được sử dụng để tokenize là Byte-pair-encoding (BPE).
- **character_coverage=1.0:** Ép tokenizer sử dụng toàn bộ ký tự trong tập train cho bộ từ điển.
- **pad_id=0, unk_id=1, bos_id=2, eos_id=3:** Gán ID cho các token đặc biệt trong bài, bao gồm <pad>, <unk>, <bos>, <eos>, lần lượt mang ID 0, 1, 2 và 3.

Nhận xét: Việc lựa chọn BPE cho bài toán là hợp lý, do BPE phù hợp với cả tiếng Trung và tiếng Việt.

```

1 class TranslationDataset(Dataset):
2     """Dataset for Chinese-Vietnamese translation pairs."""
3
4     def __init__(self, src, tgt, sp_src, sp_tgt, max_len=MAX_LEN):
5         self.src = src
6         self.tgt = tgt
7         self.sp_src = sp_src
8         self.sp_tgt = sp_tgt
9         self.max_len = max_len
10
11     def __len__(self):
12         return len(self.src)
13
14     def __getitem__(self, idx):
15         # Add BOS (2) and EOS (3) tokens
16         src_ids = [2] + self.sp_src.EncodeAsIds(self.src[idx][:self.max_len-2] + [3]
17         tgt_ids = [2] + self.sp_tgt.EncodeAsIds(self.tgt[idx][:self.max_len-2] + [3]
18         return torch.tensor(src_ids), torch.tensor(tgt_ids)
19
20     def collate_fn(batch):
21         """Collate function to pad sequences to the same length."""
22         srcs, tgts = zip(*batch)
23         max_src = max(len(s) for s in srcs)
24         max_tgt = max(len(t) for t in tgts)
25
26         # Pad with 0 (PAD token)
27         src_pad = torch.zeros(len(batch), max_src, dtype=torch.long)
28         tgt_pad = torch.zeros(len(batch), max_tgt, dtype=torch.long)
29
30         for i, (s, t) in enumerate(zip(srcs, tgts)):
31             src_pad[i, :len(s)] = s
32             tgt_pad[i, :len(t)] = t
33
34         return src_pad, tgt_pad
35
36     # Create training dataset and dataloader
37     # Split data: 90% train, 10% validation
38     total_samples = len(train_src)
39     train_size = int(0.9 * total_samples)
40
41     train_src_split = train_src[:train_size]
42     train_tgt_split = train_tgt[:train_size]
43     valid_src = train_src[train_size:]
44     valid_tgt = train_tgt[train_size:]
45
46     dataset = TranslationDataset(train_src_split, train_tgt_split, sp_zh, sp_vi)
47     dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, collate_fn=
48                             collate_fn, num_workers=4, pin_memory=True)
49
49     valid_dataset = TranslationDataset(valid_src, valid_tgt, sp_zh, sp_vi)

```

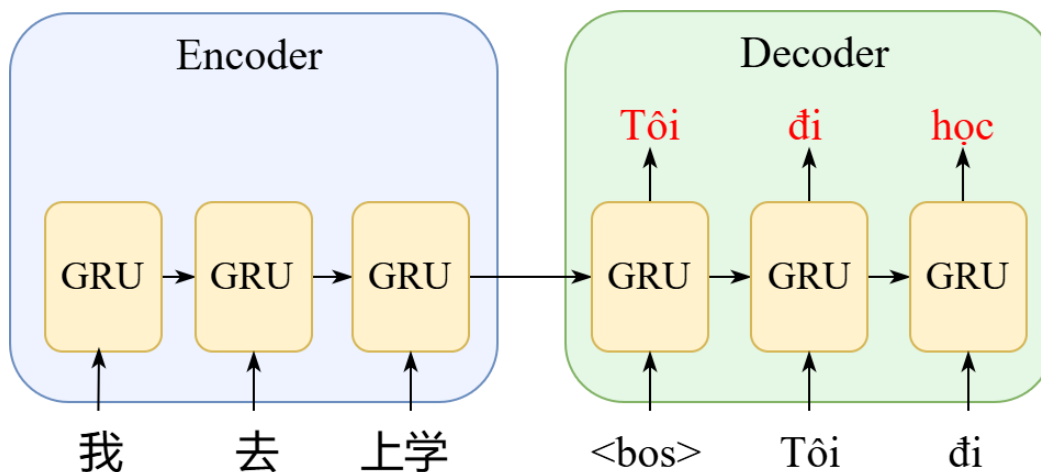
```
50 valid_loader = DataLoader(valid_dataset, batch_size=32, shuffle=False, collate_fn=
                                collate_fn, num_workers=2, pin_memory=True)
```

Giải thích: Đoạn code trên sẽ xây dựng bộ dữ liệu

- **Phân chia tập train và val:** Tập train và val được xây dựng từ tập train ban đầu của bạn tổ chức, chia ra với tỉ lệ **9:1**.
- **TranslationDataset:** Trước khi đưa vào mô hình, mỗi sample sẽ được bổ sung token **<bos>** và **<eos>** lần lượt báo hiệu cho mô hình rằng đây là khởi đầu và kết thúc của câu. Đây là cách xử lý tiêu chuẩn cho các bài toán NLP.
- **collate_fn:** Do mô hình cần các sample có kích thước giống nhau là **MAX_LEN**, ta cần điều chỉnh lại độ dài mỗi sample trước khi đưa vào mô hình. Với các câu quá dài (lớn hơn **MAX_LEN**), ta bỏ đi phần phía sau. Với các câu ngắn (nhỏ hơn **MAX_LEN**), ta điều chỉnh độ dài bằng cách thêm các token **<pad>** vào.

Nhận xét: Phần xử lý dữ liệu của baseline tương đối đơn giản. Ta cũng nhận thấy rằng baseline không sử dụng kỹ thuật tiền xử lý hay augmentation nào khác. Đây cũng là một phần hạn chế của baseline, khi tập train có kích thước không quá lớn, có thể khiến cho mô hình không có khả năng tổng quát tốt.

Bước 3: Model Architecture



Hình 4: Mô hình Seq2Seq của baseline

```
1 class EncoderRNN(nn.Module):
2     """GRU-based encoder."""
3
4     def __init__(self, vocab_size, emb_size, hidden_size):
5         super().__init__()
6         self.embedding = nn.Embedding(vocab_size, emb_size, padding_idx=0)
```

```

7         self.dropout = nn.Dropout(0.3)
8         self.rnn = nn.GRU(emb_size, hidden_size, batch_first=True)
9
10        def forward(self, src):
11            emb = self.dropout(self.embedding(src))
12            _, hidden = self.rnn(emb)
13            return hidden
14
15
16        class DecoderRNN(nn.Module):
17            """GRU-based decoder with teacher forcing."""
18
19            def __init__(self, vocab_size, emb_size, hidden_size):
20                super().__init__()
21                self.embedding = nn.Embedding(vocab_size, emb_size, padding_idx=0)
22                self.dropout = nn.Dropout(0.3)
23                self.rnn = nn.GRU(emb_size, hidden_size, batch_first=True)
24                self.fc = nn.Linear(hidden_size, vocab_size)
25
26            def forward(self, input_step, hidden):
27                emb = self.dropout(self.embedding(input_step))
28                output, hidden = self.rnn(emb, hidden)
29                pred = self.fc(output.squeeze(1))
30                return pred, hidden
31
32
33        class Seq2Seq(nn.Module):
34            """Sequence-to-sequence model."""
35
36            def __init__(self, encoder, decoder):
37                super().__init__()
38                self.encoder = encoder
39                self.decoder = decoder
40
41            def forward(self, src, tgt, teacher_forcing_ratio=0.3):
42                batch_size = src.size(0)
43                tgt_len = tgt.size(1)
44                vocab_size = self.decoder.fc.out_features
45
46                # Store outputs
47                outputs = torch.zeros(batch_size, tgt_len, vocab_size).to(src.device)
48
49                # Encode source sentence
50                hidden = self.encoder(src)
51
52                # Start with BOS token
53                input_step = tgt[:, 0].unsqueeze(1)
54
55                # Decode step by step
56                for t in range(1, tgt_len):
57                    pred, hidden = self.decoder(input_step, hidden)
58                    outputs[:, t] = pred
59
60                # Teacher forcing

```

```

61         teacher_force = random.random() < teacher_forcing_ratio
62         input_step = tgt[:, t].unsqueeze(1) if teacher_force else pred.argmax(1).
            unsqueeze(1)
63
64         return outputs
65
66     # Initialize model
67     encoder = EncoderRNN(sp_zh.GetPieceSize(), EMB_SIZE, HID_SIZE)
68     decoder = DecoderRNN(sp_vi.GetPieceSize(), EMB_SIZE, HID_SIZE)
69     model = Seq2Seq(encoder, decoder).to(DEVICE)

```

Giải thích: Baseline sử dụng mô hình Seq2Seq với các khối Encoder - Decoder, với tổng cộng 919,992 tham số, được xây dựng từ GRU:

- **Encoder:** Xử lý tuần tự các ký tự đầu vào. Hidden state cuối cùng của Encoder đóng vai trò như một *context vector*, chứa thông tin ngữ nghĩa tổng hợp của toàn bộ câu đầu vào
- **Decoder:** sử dụng hidden state này để sinh ra dự đoán. Tại mỗi bước, Decoder GRU nhận vào các thông tin trước đó, bao gồm token được sinh ra từ mô hình hoặc từ dữ liệu huấn luyện, cùng với hidden state, để sinh ra phân phối xác suất dự đoán từ tiếp theo.
- **Dropout:** Khối dropout được sử dụng trong cả **Encoder** và **Decoder**. Đây là kỹ thuật regularization có thể giúp mô hình tổng quát tốt hơn.
- **Teacher forcing:** Kỹ thuật này được thể hiện qua việc mô hình chọn token được sinh ra trong decoder hoặc token trong tập train.

Nhận xét: Việc lựa chọn GRU làm cốt lõi cho mô hình có nhiều hạn chế. Mặc dù GRU được đánh giá là tốt hơn RNN trong các bài toán NLP, GRU vẫn kém cạnh so với các mô hình Transformer trong việc nắm bắt ngữ nghĩa của câu, đặc biệt trong các câu dài. Ngoài ra, kích thước của mô hình tương đối nhỏ (chưa đến 1M tham số). Do đó, một cải tiến có thể thực hiện là nâng cấp mô hình.

Bước 4: Training

```

1  def train_epoch(model, dataloader, criterion, optimizer):
2      """Train for one epoch."""
3      model.train()
4      total_loss = 0
5
6      for src, tgt in dataloader:
7          src, tgt = src.to(DEVICE), tgt.to(DEVICE)
8
9          optimizer.zero_grad()
10         output = model(src, tgt)
11

```

```

12     # Calculate loss (ignore first BOS token)
13     loss = criterion(
14         output[:, 1:].reshape(-1, output.size(-1)),
15         tgt[:, 1:].reshape(-1)
16     )
17
18     loss.backward()
19     torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
20     optimizer.step()
21
22     total_loss += loss.item()
23
24     return total_loss / len(dataloader)
25
26
27 @torch.no_grad()
28 def evaluate_bleu(model, dataloader, sp_tgt):
29     """Evaluate model using SacreBLEU metric."""
30     model.eval()
31     hyps, refs = [], []
32
33     pbar = tqdm(dataloader, desc="Evaluating", leave=False)
34     for src, tgt in pbar:
35         src = src.to(DEVICE)
36
37         # Encode
38         hidden = model.encoder(src)
39
40         # Decode (greedy)
41         input_step = torch.full((src.size(0), 1), 2, dtype=torch.long, device=DEVICE)
42         decoded = [[] for _ in range(src.size(0))]
43
44         for _ in range(MAX_LEN):
45             pred, hidden = model.decoder(input_step, hidden)
46             next_token = pred.argmax(1).unsqueeze(1)
47
48             for i in range(src.size(0)):
49                 decoded[i].append(next_token[i].item())
50
51             input_step = next_token
52
53         # Convert to text
54         for i in range(src.size(0)):
55             ids = decoded[i]
56             if 3 in ids: # Stop at EOS
57                 ids = ids[:ids.index(3)]
58             hyps.append(sp_tgt.DecodeIds(ids))
59
60             ref_ids = tgt[i].tolist()[1:-1] # Remove BOS and EOS
61             refs.append(sp_tgt.DecodeIds([x for x in ref_ids if x not in [0, 1]]))
62
63     bleu = sacrebleu.corpus_bleu(hyps, [refs])
64     return bleu.score
65

```

```

66 # Loss function and optimizer
67 criterion = nn.CrossEntropyLoss(ignore_index=0) # Ignore padding
68 optimizer = optim.Adam(model.parameters(), lr=LR)
69
70 print("Starting training...\n")
71
72 # Training loop with best model saving
73 best_bleu = 0.0
74 best_model_path = os.path.join(SAVE_DIR, "best_model.pt")
75
76 for epoch in range(1, EPOCHS + 1):
77     loss = train_epoch(model, dataloader, criterion, optimizer)
78     bleu = evaluate_bleu(model, valid_loader, sp_vi)
79
80     # Save best model
81     if bleu > best_bleu:
82         best_bleu = bleu
83         torch.save({
84             "epoch": epoch,
85             "model_state_dict": model.state_dict(),
86             "optimizer_state_dict": optimizer.state_dict(),
87             "bleu": bleu,
88             "loss": loss
89         }, best_model_path)
90         print(f"Epoch {epoch:02d} | Loss={loss:.3f} | SacreBLEU={bleu:.2f} Best model
91               saved!")
92     else:
93         print(f"Epoch {epoch:02d} | Loss={loss:.3f} | SacreBLEU={bleu:.2f}")

```

Giải thích: Các chiến thuật training trong baseline gồm:

- Hàm loss: Cross Entropy.
- Optimizer: Adam.

Nhận xét: Nhìn chung, lựa chọn hàm loss và optimizer của baseline có thể được coi là tiêu chuẩn trong bài toán MT. Tuy nhiên, ngoài ra, baseline cũng không sử dụng các kỹ thuật training nào khác như learning rate scheduler, Label Smoothing,... Đây là một thứ mà chúng ta có thể cải thiện.

Bước 5: Decode

```

1 @torch.no_grad()
2 def translate_test(model, test_src, sp_src, sp_tgt, out_path):
3     """Translate test set and save to CSV."""
4     model.eval()
5     outputs = []
6
7     for s in tqdm(test_src, desc="Translating"):
8         # Tokenize source sentence

```

```

9      src_ids = [2] + sp_src.EncodeAsIds(s)[:MAX_LEN-2] + [3]
10     src_tensor = torch.tensor(src_ids, dtype=torch.long, device=DEVICE).unsqueeze(0)
11
12     # Encode
13     hidden = model.encoder(src_tensor)
14
15     # Decode
16     input_step = torch.full((1, 1), 2, dtype=torch.long, device=DEVICE)
17     decoded = []
18
19     for _ in range(MAX_LEN):
20         pred, hidden = model.decoder(input_step, hidden)
21         next_token = pred.argmax(1)
22         token = next_token.item()
23
24         if token == 3: # EOS
25             break
26
27         decoded.append(token)
28         input_step = next_token.unsqueeze(1)
29
30     # Decode to Vietnamese text
31     vi_sent = sp_tgt.DecodeIds(decoded)
32     outputs.append(vi_sent)
33
34     # Save to CSV
35     df = pd.DataFrame({"tieng_trung": test_src, "tieng_viet": outputs})
36     df.to_csv(out_path, index=False, encoding="utf-8-sig")
37
38     return out_path

```

Giải thích: Sau khi hoàn thành việc luyện mô hình, baseline tiếp tục với phần inference trên các dữ liệu test, trong đó chiến thuật decode được sử dụng là Greedy Decode: Mô hình sẽ lần lượt chọn từ có xác suất cao nhất để dự đoán cho từ tiếp theo.

Nhận xét: Kỹ thuật decode này đơn giản và khá yếu so với những kỹ thuật khác như **Beam search**.

Nhận xét chung về baseline

Baseline đã thực hiện đầy đủ các quá trình của một bài toán MT. Tuy nhiên, nó vẫn còn nhiều hạn chế:

- **Xử lý dữ liệu:** Ngoài việc sử dụng BPE tokenization, baseline không sử dụng các kỹ thuật nào để xử lý dữ liệu. Kích thước của tập dữ liệu không quá lớn (xấp xỉ 32000 samples), mô hình có thể không tổng quát tốt đối với tập test. Do đó, việc bổ sung các kỹ thuật tiền xử lý dữ liệu hoặc augmentation là vô cùng cần thiết.
- **Mô hình:** Mô hình baseline Seq2Seq sử dụng GRU đơn giản và dễ triển khai. Thế nhưng, nó vẫn tồn tại nhiều hạn chế. GRU tuy tốt hơn RNN khi xử lý các văn bản có kích thước

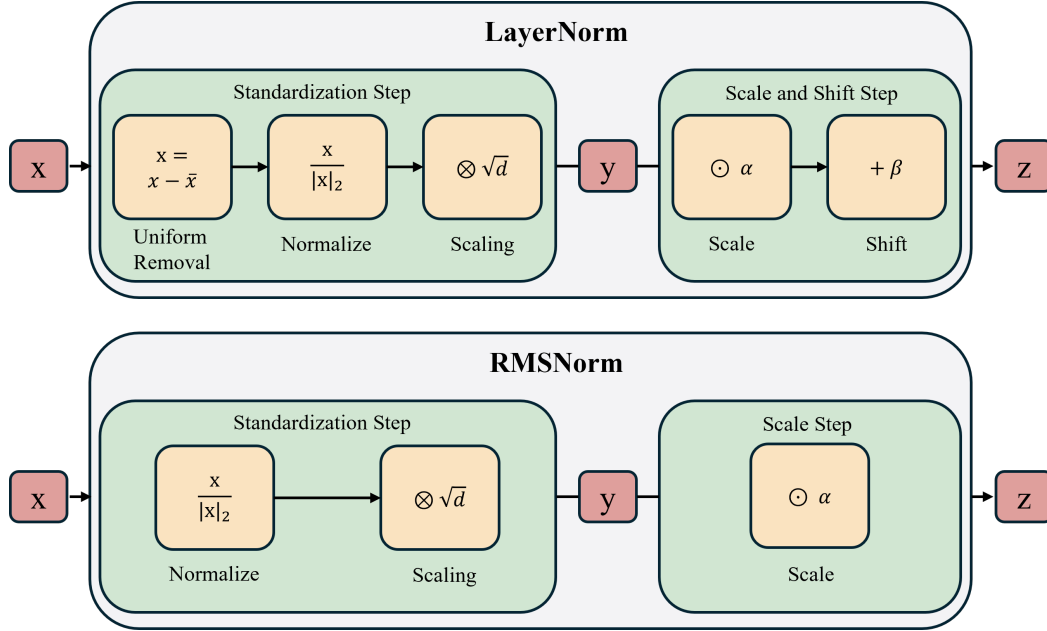
dài nhưng vẫn yếu hơn so với các mô hình như Transformer, vốn được sử dụng khá nhiều hiện nay trong các bài toán MT. Ngoài ra, kích thước mô hình (VOCAB_SIZE, EMB_SIZE, HID_SIZE) vẫn còn tương đối nhỏ. Do đó, để cải thiện baseline, chúng ta có thể thử sử dụng các mô hình có kiến trúc mạnh mẽ và khả năng học tốt hơn.

- **Chiến thuật training:** Baseline đang huấn luyện mô hình với Cross Entropy loss và Adam optimizer. Mặc dù đây là lựa chọn tốt và phổ biến, hiện nay đã có nhiều chiến thuật training khác có thể được áp dụng như AdamW, learning rate scheduler, label smoothing, regularization, để cải thiện mô hình.
- **Chiến thuật decode:** Baseline đang sử dụng Greedy decode. So với các phương pháp Sampling hiện nay như Beam search, Greedy Decode vẫn còn nhiều hạn chế. Đôi khi Greedy Decode có thể dẫn đến những câu vô nghĩa ảnh hưởng đến kết quả

Điểm SacreBLEU của baseline trên tập public_test là 4.52, tương đối thấp. Từ những nhận xét trên, chúng ta có thể cải thiện baseline ở nhiều khía cạnh (xử lý dữ liệu, thay đổi kiến trúc mô hình, thay đổi kỹ thuật huấn luyện, decode,...).

IV. Bổ sung kiến thức về Transformer

IV.1. RMSNorm



Hình 5: Minh họa LayerNorm và RMSNorm.

Trong kiến trúc Transformer gốc, *LayerNorm* chuẩn hoá một vector kích hoạt $\mathbf{x} \in \mathbb{R}^d$ bằng cách trừ đi trung bình theo chiều kênh rồi chia cho độ lệch chuẩn. Cụ thể, đầu ra được tính theo

$$y = \alpha \odot \frac{\mathbf{x} - \mu(\mathbf{x})}{\sqrt{\sigma^2(\mathbf{x}) + \varepsilon}} + \beta.$$

Cách làm này bảo đảm mỗi chiều có phân phối gần chuẩn, nhưng đồng thời làm thay đổi cả hướng lẫn độ lớn của vector, và đòi hỏi tính cả mean lẫn variance cho từng bước lan truyền.

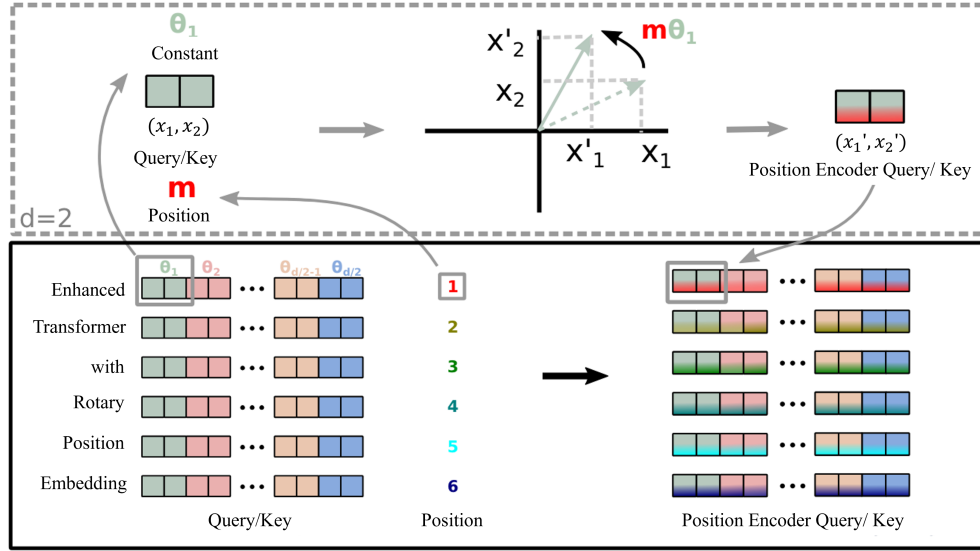
RMSNorm loại bỏ bước trừ mean và chỉ chuẩn hoá theo *chuẩn RMS* của vector:

$$y = \alpha \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})}, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \varepsilon}.$$

Nhờ vậy, RMSNorm giữ nguyên hướng của $\mathbf{x}$, chỉ điều chỉnh độ lớn, đồng thời giảm bớt phép tính (không cần variance và bias β), dẫn tới chi phí tính toán thấp hơn và ít nhiễu số hơn trên các mô hình rất sâu.

Trong các LLM hiện đại (như LLaMA, Qwen, v.v.), RMSNorm thường được ưa dùng vì giúp *ổn định gradient* khi kết hợp với sơ đồ *pre-norm* và residual connection, cải thiện độ ổn định huấn luyện ở kích thước lớp ẩn lớn, và thực nghiệm cho thấy có thể đạt chất lượng tương đương hoặc tốt hơn LayerNorm với chi phí tính toán đơn giản hơn.

IV.2. RoPE



Hình 6: Minh họa cách hoạt động của RoPE.

Trong Transformer gốc, thông tin vị trí thường được đưa vào bằng *positional embedding tuyệt đối*, ví dụ dạng hình sin-cos: một vector $PE(m) \in \mathbb{R}^d$ phụ thuộc vào chỉ số vị trí m được cộng trực tiếp vào biểu diễn token:

$$\mathbf{x}'_m = \mathbf{x}_m + PE(m).$$

Cách làm này giúp mô hình phân biệt thứ tự các token, nhưng sự phụ thuộc vào vị trí tuyệt đối khiến việc ngoại suy sang độ dài chuỗi lớn hơn hoặc mô hình hoá quan hệ tương đối (khoảng cách $m - n$) trở nên kém tự nhiên.

Rotary Position Embedding (RoPE) thay vì cộng một vector vị trí, sẽ mã hoá vị trí bằng cách xoay các cặp chiều trong không gian 2D phụ thuộc vào chỉ số m . Cho $\mathbf{q}_m, \mathbf{k}_m \in \mathbb{R}^d$ là query và key tại vị trí m , với mỗi cặp chỉ số $(2i, 2i + 1)$ ta định nghĩa góc θ_i và áp dụng phép quay:

$$\begin{aligned} \tilde{q}_m^{(2i)} &= q_m^{(2i)} \cos(m\theta_i) - q_m^{(2i+1)} \sin(m\theta_i), \\ \tilde{q}_m^{(2i+1)} &= q_m^{(2i)} \sin(m\theta_i) + q_m^{(2i+1)} \cos(m\theta_i), \end{aligned}$$

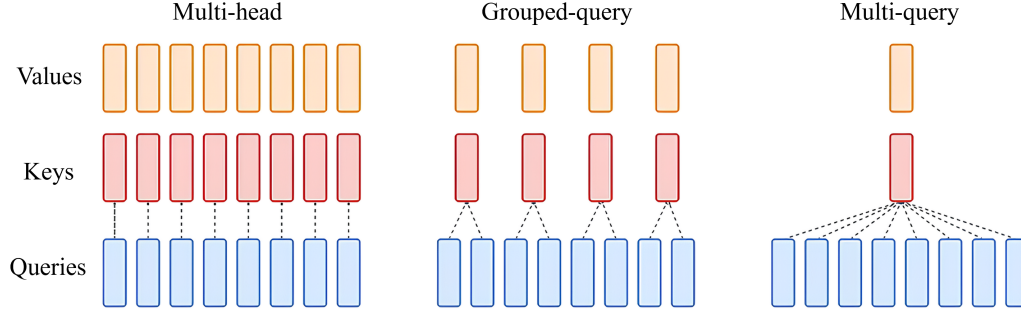
và tương tự cho $\tilde{\mathbf{k}}_n$. Khi đó, tích vô hướng trong attention trở thành

$$\tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n,$$

một hàm phụ thuộc chủ yếu vào *chênh lệch vị trí* $m - n$, nhưng thông tin vị trí tương đối trực tiếp vào cơ chế attention mà không cần sửa lại công thức softmax.

Nhờ đặc tính “vị trí tương đối hoá trong tích vô hướng”, RoPE thường cho khả năng *ngoại suy chiều dài chuỗi* tốt hơn so với positional embedding thuần túy cộng-thêm, và tương thích tự nhiên với cơ chế scaled dot-product attention. Thực nghiệm trên nhiều mô hình lớn (LLaMA, Qwen, v.v.) cho thấy RoPE giúp mô hình ổn định hơn trên ngữ cảnh dài, tận dụng tốt hơn thông tin trật tự, nên ngày càng trở thành lựa chọn mặc định cho các LLM hiện đại.

IV.3. Grouped-query attention



Hình 7: Mô phỏng 3 dạng attention cơ bản: multi-head, grouped-query và multi-query.

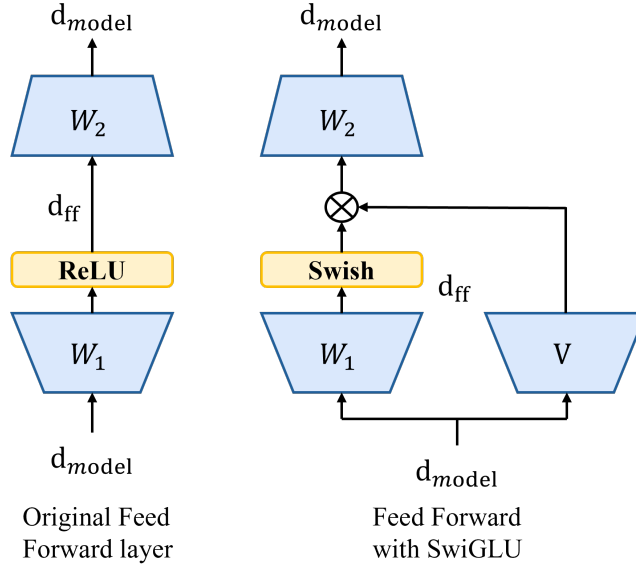
Trong Transformer chuẩn, *multi-head attention* sử dụng h head với các ma trận riêng cho query, key và value: mỗi head i có $(W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}) \in \mathbb{R}^{d_{\text{model}} \times d_h}$. Ví dụ, với $h = 16$ và chiều mỗi head d_h , ta có đủ 16 cặp (K, V) khác nhau, giúp mô hình học được nhiều kiểu quan hệ song song, nhưng chi phí lưu trữ KV cache trong giai đoạn suy luận tỉ lệ với $16 \cdot L \cdot d_h$, rất tốn bộ nhớ khi L và d_{model} lớn.

Multi-query attention (MQA) là một cực trị theo hướng tối ưu hiệu năng: thay vì có 16 cặp (K, V) , ta chỉ dùng *một* cặp (K, V) chung cho tất cả head (tức $h_{kv} = 1$). Khi đó, chi phí KV cache giảm xuống còn $\sim 1 \cdot L \cdot d_h$, giúp tăng tốc decoding đáng kể. Tuy nhiên, vì mọi head cùng nhìn vào một không gian key-value duy nhất, mức độ đa dạng hoá biểu diễn có thể suy giảm, và một số mô hình lớn ghi nhận giảm nhẹ về chất lượng khi dùng MQA thuần túy.

Grouped-query attention (GQA) được đề xuất như một điểm cân bằng giữa MHA và MQA. Ta chọn một số nhóm key-value h_{kv} sao cho $1 < h_{kv} < h$, ví dụ $h = 16$ nhưng $h_{kv} = 4$. Khi đó, 16 head query được chia thành 4 nhóm, mỗi nhóm 4 head chia sẻ chung một cặp (K, V) , nên tổng số cặp (K, V) còn 4 thay vì 16 (giảm KV cache xuống $\sim 4 \cdot L \cdot d_h$), trong khi vẫn giữ đủ số head query để mô hình hoá nhiều pattern attention khác nhau.

Nhờ giảm đáng kể chi phí lưu trữ và truy cập KV cache (gần với MQA) nhưng vẫn duy trì độ đa dạng không gian key-value tốt hơn, GQA thường cho chất lượng dịch/generative gần ngang multi-head truyền thống, với tốc độ suy luận và footprint bộ nhớ tương đương hoặc tiệm cận MQA. Đây là lý do nhiều LLM hiện đại (chẳng hạn LLaMA 2, Mistral, Qwen, v.v.) ưu tiên dùng *grouped-query attention* trong self-attention và/hoặc cross-attention, đặc biệt trong các ứng dụng yêu cầu serving thời gian thực và context dài.

IV.4. SwiGLU



Hình 8: Mô phỏng hai dạng FFN thường gặp.

Trong Transformer gốc, tầng *feed-forward* (FFN) thường có dạng hai lớp tuyến tính kèm hàm kích hoạt, mở rộng chiều ẩn từ d_{model} lên d_{ff} rồi thu hẹp trở lại:

$$\text{FFN}(\mathbf{x}) = W_2 \phi(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2,$$

trong đó ϕ thường là ReLU hoặc GELU. Cấu trúc này đơn giản nhưng ReLU có vùng chết (output bằng 0 cho giá trị âm) và không có cơ chế *gating*, nên đôi khi kém linh hoạt khi mô hình hoá các tương tác phi tuyến tính vì trong LLM lớn.

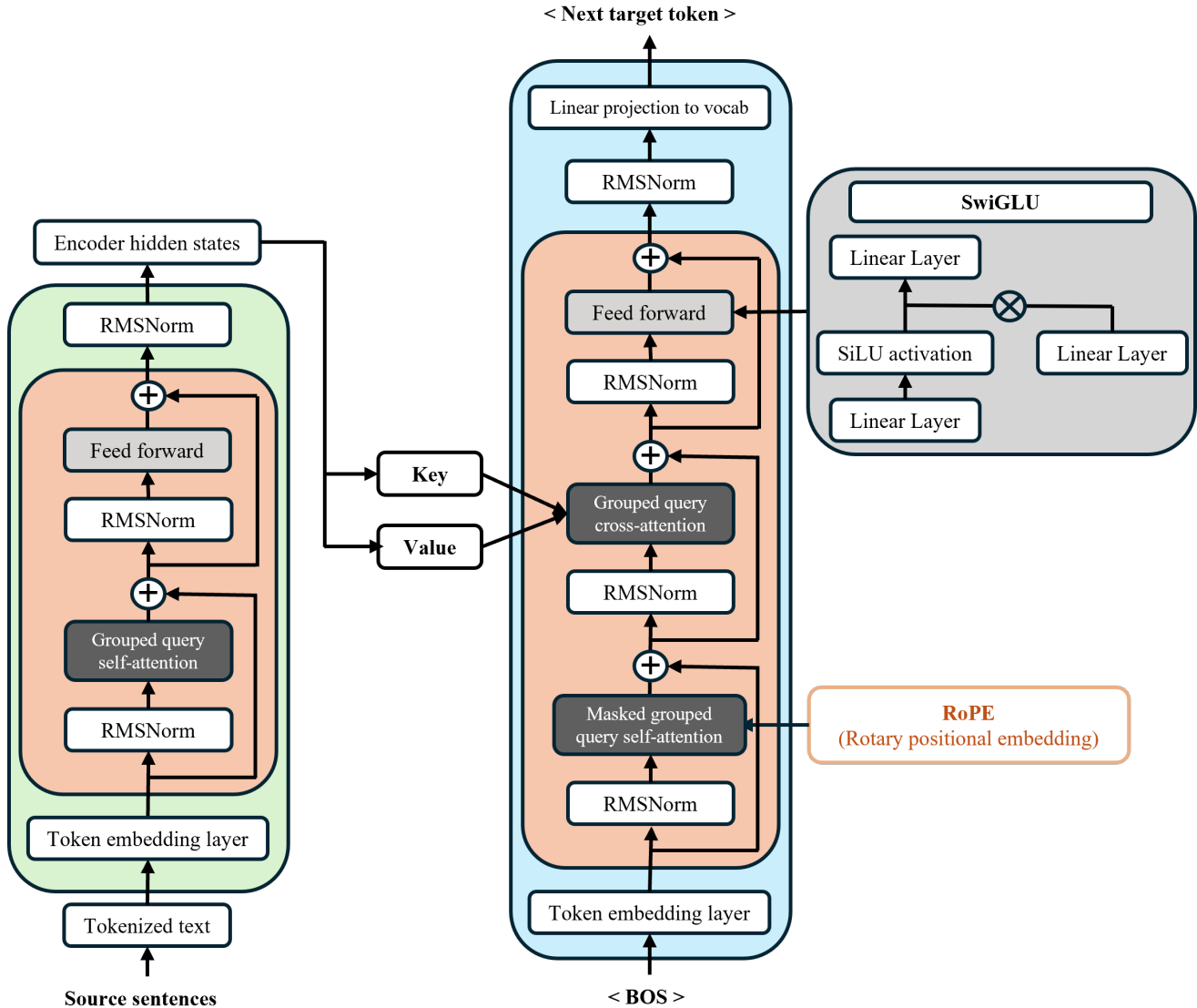
SwiGLU kết hợp ý tưởng *gating* (như GLU) với hàm kích hoạt trơn *Swish*/*SiLU*. Cho hai phép chiếu tuyến tính $W_1, V \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, ta định nghĩa:

$$\begin{aligned} \mathbf{u} &= W_1 \mathbf{x}, & \mathbf{v} &= V \mathbf{x}, \\ \text{SwiGLU}(\mathbf{x}) &= W_2 (\text{SiLU}(\mathbf{u}) \odot \mathbf{v}), \end{aligned}$$

trong đó $\text{SiLU}(t) = t \sigma(t)$ với $\sigma(t)$ là hàm sigmoid, và $\odot$ là phép nhân từng phần tử. Nhánh $\text{SiLU}(\mathbf{u})$ đóng vai trò *gate* trơn, quyết định kênh nào được khuếch đại hay bị triệt tiêu, trong khi nhánh $\mathbf{v}$ mang nội dung tuyến tính được điều biến bởi gate này.

Nhờ vừa có hàm kích hoạt trơn (giảm nguy cơ “chết” neuron như ReLU), vừa có cơ chế *gating* giàu biểu đạt, SwiGLU thường cho gradient ổn định hơn và khả năng mô hình hoá tốt hơn trong tầng FFN. Nhiều LLM hiện đại (PaLM, LLaMA, v.v.) đã chuyển từ FFN dùng ReLU/GELU sang SwiGLU và quan sát được cải thiện rõ rệt về perplexity và chất lượng sinh, đặc biệt ở thiết lập mô hình lớn với chiều d_{ff} rất cao.

IV.5. Kiến trúc mô hình Transformer Encoder-Decoder được sử dụng



Hình 9: Kiến trúc mô hình được sử dụng trong bài toán dịch máy.

Mô hình được triển khai là một *Transformer Encoder-Decoder* với RMSNorm, Grouped-query attention kết hợp RoPE và tầng FFN dùng SwiGLU. Dưới đây là mô tả chi tiết vai trò của từng lớp trong quá trình lan truyền tiến.

Trong Encoder: Chuỗi nguồn được ánh xạ sang không gian liên tục thông qua *token embedding* và sau đó đi qua nhiều lớp encoder.

- **RMSNorm (pre-norm):** chuẩn hoá biên độ của các vector biểu diễn trước khi đưa vào self-attention, giúp ổn định gradient và duy trì độ lớn tín hiệu khi mô hình sâu.

- **Grouped-query self-attention with RoPE:** cho phép mỗi vị trí quan sát toàn bộ chuỗi nguồn để thu nhận ngữ cảnh toàn cục. RoPE mã hoá quan hệ vị trí tương đối trực tiếp trong không gian attention, còn grouped-query attention giảm chi phí lưu trữ và truy cập key-value mà vẫn giữ nhiều hướng truy vấn (query heads).
- **Residual connection:** giữ lại thông tin tầng trước, hỗ trợ mô hình hội tụ dễ hơn và tránh mất mát thông tin gốc.
- **RMSNorm:** chuẩn hoá lại tín hiệu sau attention nhằm bảo đảm độ ổn định trước khi đi vào tầng phi tuyến.
- **FFN với SwiGLU:** thực hiện các biến đổi phi tuyến độc lập trên từng vị trí. Cơ chế gating của SwiGLU cho phép mô hình chọn lọc các chiều quan trọng, tăng khả năng biểu đạt so với FFN truyền thống.
- **Residual connection:** duy trì dòng thông tin ổn định xuyên suốt nhiều lớp encoder.

Sau khi đi qua toàn bộ encoder, mô hình thu được bộ *contextual representations* giàu ngữ nghĩa, được sử dụng làm bộ nhớ cho decoder.

Trong Decoder: Chuỗi đích được shift-right để tạo đầu vào theo cơ chế autoregressive và đi qua nhiều lớp decoder có ba thành phần chính.

- **RMSNorm (pre-norm):** chuẩn bị tín hiệu ổn định trước khi vào self-attention.
- **Grouped-query masked self-attention with causal mask:** mô hình hoá phân bố ngôn ngữ của phía đích dưới điều kiện autoregressive. Causal mask đảm bảo mỗi vị trí chỉ được nhìn các token xuất hiện trước đó trong chuỗi.
- **Residual connection:** giữ thông tin trước attention và hỗ trợ lan truyền ổn định.
- **RMSNorm:** chuẩn hoá biểu diễn để chuẩn bị cho bước tương tác với encoder.
- **Grouped-query cross-attention:** cho phép decoder truy vấn lên toàn bộ encoder representations. Đây là nơi mô hình học cách căn chỉnh (alignment) giữa nguồn và đích để quyết định từ đích phù hợp.
- **Residual connection:** duy trì sự ổn định của dòng thông tin sau cross-attention.
- **RMSNorm:** chuẩn hoá trước khi áp dụng tầng phi tuyến.
- **FFN với SwiGLU:** tinh chỉnh biểu diễn tại mỗi vị trí đích thông qua biến đổi phi tuyến mạnh, tăng khả năng mô hình hoá đặc trưng ngữ nghĩa phức tạp.
- **Residual connection:** hỗ trợ huấn luyện sâu và giữ thông tin nhất quán.

Tầng sinh đầu ra: Biểu diễn cuối của decoder được chuẩn hoá và chiếu lên không gian từ vựng thông qua trọng số embedding dùng chung, tạo thành logits cho token tiếp theo. Trong huấn luyện, các logits này được tối ưu bằng cross-entropy có label smoothing; trong suy luận, chúng được sử dụng để sinh chuỗi đích theo kiểu autoregressive.

V. Mô hình Transformer Encoder-Decoder cho bài toán dịch thuật Hoa-Việt

Trong phần này, Mô hình dịch thuật Hoa-Việt được triển khai dựa trên kiến trúc *Transformer Encoder-Decoder* hiện đại, kết hợp các cải tiến như RMSNorm, RoPE, Grouped-query attention và tầng FFN-SwiGLU. Cấu trúc này cho phép mô hình nắm bắt ngữ cảnh song ngữ hiệu quả hơn, ổn định trong huấn luyện và đạt kết quả tốt dựa trên đánh giá của ban tổ chức.

Bước 1: Import thư viện và tải dữ liệu

```
1 !gdown 1KuDLpUaSe0wzDWEhgG0wKXV1_AVmzLK4
2 !unzip ./dataset.zip -d .
3
4 import os
5 import random
6 import sentencepiece as spm
7 import pandas as pd
8 from tqdm import tqdm
9 import matplotlib.pyplot as plt
10 import numpy as np
11 import math
12 from dataclasses import dataclass
13 from typing import List, Tuple, Any, Dict, Optional
14
15 import torch
16 import torch.nn as nn
17 import torch.nn.functional as F
18 from torch.utils.data import Dataset, DataLoader, SubsetRandomSampler
19 import torch.optim as optim
20 import sacrebleu
```

Trong bước đầu tiên, tập dữ liệu được tải về từ Google Drive bằng `gdown` và giải nén ra thư mục làm việc. Tiếp theo, các thư viện cần thiết được import, bao gồm: `pandas`, `numpy`, `matplotlib` và `tqdm` cho xử lý và quan sát dữ liệu; `sentencepiece` cho việc xây dựng tokenizer theo kiểu subword; các module `torch`, `torch.nn`, `Dataset`, `DataLoader` và `optim` cho việc xây dựng và huấn luyện mô hình Transformer; cùng với `sacrebleu` dùng để đánh giá chất lượng dịch máy thông qua chỉ số BLEU.

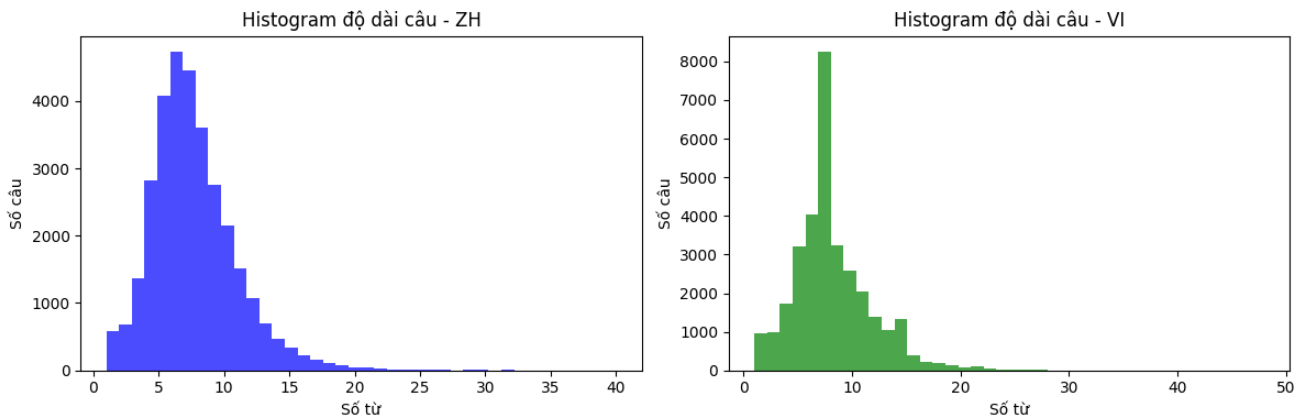
Bước 2: Khám phá dữ liệu song ngữ

```

1 BASE_DIR = "."
2 TRAIN_DIR = os.path.join(BASE_DIR, "dataset", "train")
3 SRC_FILE = os.path.join(TRAIN_DIR, "train.zh")
4 TGT_FILE = os.path.join(TRAIN_DIR, "train.vi")
5 TOKENIZER_DIR = os.path.join(BASE_DIR, "tokenizer_train32")
6 TOKENIZER_PREFIX = os.path.join(TOKENIZER_DIR, "spm_zh_vi_joint")
7 MAX_TOKENS = 32
8 VOCAB_SIZE = 8000
9 USER_SYMBOLS = ["<2zh>", "<2vi>"]
10
11 os.makedirs(TOKENIZER_DIR, exist_ok=True)
12
13 # ----- LOAD -----
14 def load_lines(path):
15     with open(path, "r", encoding="utf-8") as f:
16         return [l.strip() for l in f if l.strip()]
17
18 zh_lines = load_lines(SRC_FILE)
19 vi_lines = load_lines(TGT_FILE)
20 #Xem thêm ở phần phụ lục

```

Trong bước này, các câu Hoa-Việt thô được nạp từ hai tệp `train.zh` và `train.vi`, kiểm tra lại số lượng để đảm bảo tương ứng 1-1 giữa hai ngôn ngữ. Một vài cặp câu đầu tiên được hiển thị nhằm quan sát nhanh chất lượng dữ liệu, đồng thời độ dài câu (tính theo số từ) được thống kê và trực quan hoá bằng histogram cho cả tiếng Hoa và tiếng Việt, giúp nắm được phân phối độ dài và chuẩn bị cho các bước cắt lọc sau này.



Hình 10: Biểu đồ số lượng từ của mỗi câu khi chưa xử lý

Bước 3: Xây dựng tokenizer và lọc độ dài câu

```

1 # ----- TRAIN SENTENCEPIECE -----
2 temp_corpus = os.path.join(TOKENIZER_DIR, "temp_corpus.txt")
3 with open(temp_corpus, "w", encoding="utf-8") as fout:
4     for zh, vi in zip(zh_lines, vi_lines):
5         fout.write(zh + "\n")
6         fout.write(vi + "\n")
7
8 spm_args = (
9     f"--input={temp_corpus} ",
10    f"--model_prefix={TOKENIZER_PREFIX} ",
11    f"--vocab_size={VOCAB_SIZE} ",
12    f"--model_type=bpe ",
13    f"--character_coverage=1.0 ",
14    "--pad_id=0 --unk_id=1 --bos_id=2 --eos_id=3 ",
15    f"--user_defined_symbols='{','.join(USER_SYMBOLS)}'"
16 )
17 spm.SentencePieceTrainer.Train(" ".join(spm_args))
18
19 print(f"Tokenizer saved to {TOKENIZER_PREFIX}.model")
20
21 # ----- LOAD TOKENIZER -----
22 sp = spm.SentencePieceProcessor()
23 sp.Load(f"{TOKENIZER_PREFIX}.model")
24
25 # ----- RELOAD RAW DATA -----
26 def load_lines(path):
27     with open(path, "r", encoding="utf-8") as f:
28         return [l.strip() for l in f if l.strip()]
29
30 zh_lines = load_lines(SRC_FILE)
31 vi_lines = load_lines(TGT_FILE)
32 assert len(zh_lines) == len(vi_lines)
33
34 MAX_TOK = 32
35
36 def tok_len(t, prefix=None):
37     if prefix:
38         t = f"{prefix} {t}"
39     return len(sp.encode(t, out_type=int))
40
41 # Token length BEFORE filtering
42 zh_before = [tok_len(z, "<2vi>") for z in zh_lines]
43 vi_before = [tok_len(v) for v in vi_lines]
44
45 # Filter sample dài
46 filtered_zh, filtered_vi = [], []
47 for z, v in zip(zh_lines, vi_lines):
48     if tok_len(z, "<2vi>") <= MAX_TOK and tok_len(v) <= MAX_TOK:
49         filtered_zh.append(z)
50         filtered_vi.append(v)
51

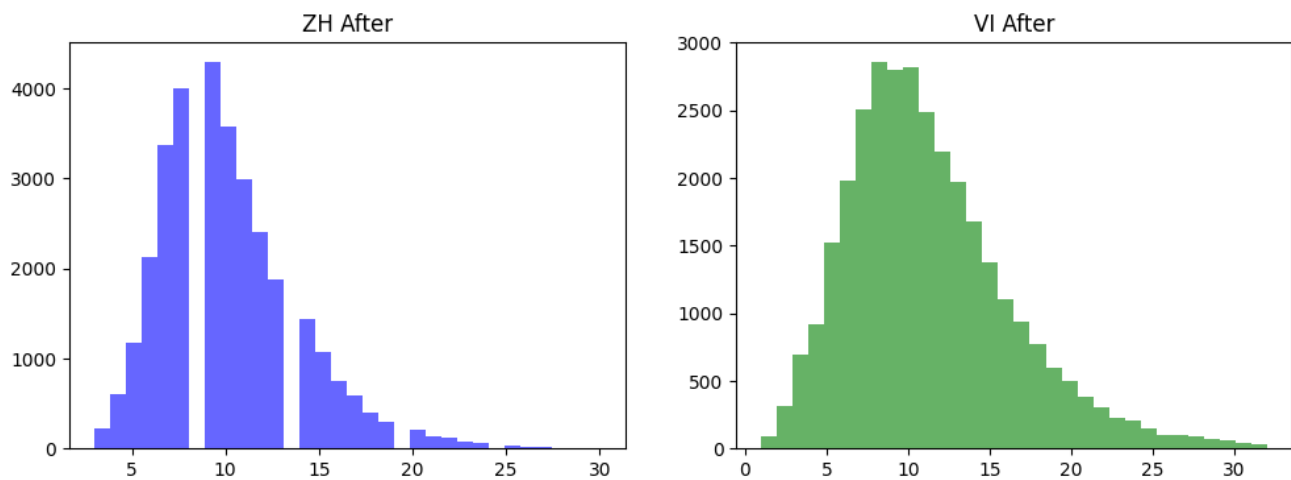
```

```

52 zh_after = np.array([tok_len(z, "<2vi>") for z in filtered_zh])
53 vi_after = np.array([tok_len(v) for v in filtered_vi])
54 #Xem thêm ở phần Phụ Lục

```

Trong bước tiếp theo, một *joint SentencePiece BPE tokenizer* được huấn luyện trên toàn bộ câu Hoa và Việt, trong đó hai token đặc biệt "<2zh>" và "<2vi>" được giữ cố định để đánh dấu ngôn ngữ nguồn. Sau khi tokenizer được huấn luyện và nạp lại, độ dài câu theo số lượng subword token được tính toán cho từng câu, và các cặp câu mà phía Hoa (kèm tiền tố "<2vi>") hoặc phía Việt vượt quá `MAX_TOK = 32` token sẽ bị loại bỏ. Histogram độ dài trước và sau khi lọc được vẽ để kiểm tra lại phân phối, trước khi lưu bộ dữ liệu sạch cuối cùng vào hai tệp `train_maxlen32.zh` và `train_maxlen32.vi` trong thư mục `./clean_data` để sử dụng cho giai đoạn huấn luyện mô hình.



Hình 11: Biểu đồ số lượng token của mỗi câu sau khi xử lý

Bước 4: Cấu hình mô hình

```

1 # Cấu hình chung cho mô hình Transformer HoaViệt
2 @dataclass
3 class RopeConfig:
4     # Đường dẫn dữ liệu và tokenizer
5     train_src_file: str
6     train_tgt_file: str
7     spm_prefix: str
8
9     # Kiến trúc Transformer
10    vocab_size: int = 8000
11    d_model: int = 768
12    n_heads: int = 12
13    n_kv_heads: int = 4

```

```

14     num_encoder_layers: int = 8
15     num_decoder_layers: int = 8
16     d_ff: int = 3072
17     dropout: float = 0.01
18     max_len: int = 32
19     rope_base: float = 10000.0
20
21     # Token đặc biệt
22     pad_token: str = "<pad>"
23     bos_token: str = "<s>"
24     eos_token: str = "</s>"
25     unk_token: str = "<unk>"
26     zh_token: str = "<2zh>"
27     vi_token: str = "<2vi>"
28
29     # Tham số huấn luyện
30     batch_size: int = 128
31     num_epochs: int = 40
32     lr_base: float = 2e-4
33     warmup_steps: int = 200
34     label_smoothing: float = 0.01
35     grad_clip: float = 1.0
36     span_mask_prob: float = 0.01
37     vi2zh_epoch_ratio: float = 0.7
38
39     # Thiết bị và hệ thống
40     device: torch.device = torch.device(
41         "cuda" if torch.cuda.is_available() else "cpu"
42     )
43     num_workers: int = 8
44     save_dir: str = "./checkpoints_bidirectional"
45     seed: int = 42

```

Lớp RopeConfig gom toàn bộ siêu tham số cốt lõi của mô hình vào một cấu trúc duy nhất: bao gồm đường dẫn đến dữ liệu song ngữ đã làm sạch và tokenizer SentencePiece, các tham số kiến trúc của Transformer (d_model, số lớp encoder/decoder, số attention heads và KV-heads, kích thước FFN, tham số RoPE), các token đặc biệt dùng trong huấn luyện, cùng các tham số huấn luyện quan trọng như batch_size, num_epochs, lr_base, warmup_steps, label_smoothing, grad_clip, tỉ lệ chọn mẫu Việt→Hoa theo từng epoch và thiết bị thực thi (cuda hoặc cpu).

Bước 5: Dataset hai chiều và lịch chọn mẫu

```

1 class BidirectionalTranslationDataset(Dataset):
2     """Sinh dữ liệu hai chiều ZHVI với span masking nhẹ."""
3
4     def __init__(
5         self,
6         src_lines: List[str],
7         tgt_lines: List[str],

```

```

8         sp_model: spm.SentencePieceProcessor,
9         config: RopeConfig,
10        is_training: bool = True,
11    ):
12        self.sp = sp_model
13        self.config = config
14        self.is_training = is_training
15
16        self.pad_id = sp_model.piece_to_id(config.pad_token)
17        self.bos_id = sp_model.piece_to_id(config.bos_token)
18        self.eos_id = sp_model.piece_to_id(config.eos_token)
19        self.unk_id = sp_model.piece_to_id(config.unk_token)
20        self.zh_id = sp_model.piece_to_id(config.zh_token)
21        self.vi_id = sp_model.piece_to_id(config.vi_token)
22
23        self.samples = []
24        if is_training:
25            # Luân phiên gán hướng zhvi và vizh
26            for i, (src, tgt) in enumerate(zip(src_lines, tgt_lines)):
27                if i % 2 == 0:
28                    self.samples.append(
29                        (self.add_lang_token(src, config.vi_token), tgt, "zh2vi")
30                    )
31                else:
32                    self.samples.append(
33                        (self.add_lang_token(src, config.zh_token), tgt, "vi2zh")
34                    )
35            else:
36                # Validation: chỉ giữ hướng zhvi
37                for src, tgt in zip(src_lines, tgt_lines):
38                    self.samples.append(
39                        (self.add_lang_token(src, config.vi_token), tgt, "zh2vi")
40                    )
41
42        self.zh2vi_indices = [
43            idx for idx, (_, _, d) in enumerate(self.samples) if d == "zh2vi"
44        ]
45        self.vi2zh_indices = [
46            idx for idx, (_, _, d) in enumerate(self.samples) if d == "vi2zh"
47        ]
48
49        def add_lang_token(self, text: str, lang_tok: str) -> str:
50            text = text.strip()
51            if text.startswith("<2vi>") or text.startswith("<2zh>"):
52                return text
53            return f"{lang_tok} {text}"
54
55        def __len__(self):
56            return len(self.samples)
57
58        def __getitem__(self, idx: int):
59            src_text, tgt_text, _ = self.samples[idx]
60            src_ids = self.sp.encode(src_text, out_type=int)[: self.config.max_len]
61            tgt_ids = [self.bos_id] + self.sp.encode(tgt_text, out_type=int) + [self.eos_id]

```

```

        ]
62     tgt_ids = tgt_ids[: self.config.max_len]
63     src_ids = self.apply_span_masking(src_ids)
64     return torch.tensor(src_ids, dtype=torch.long), torch.tensor(
65         tgt_ids, dtype=torch.long
66     )
67
68     def apply_span_masking(self, src_ids: List[int]) -> List[int]:
69         """Thay ngẫu nhiên một đoạn 12 token thường bằng <unk> với xác suất nhỏ."""
70         if not self.is_training or random.random() > self.config.span_mask_prob:
71             return src_ids
72         src_ids = src_ids.copy()
73         special = {self.pad_id, self.bos_id, self.eos_id, self.zh_id, self.vi_id}
74         maskable = [i for i, t in enumerate(src_ids) if t not in special]
75         if not maskable:
76             return src_ids
77         num_to_mask = min(random.randint(1, 2), len(maskable))
78         start = random.choice(maskable)
79         for i in range(start, min(start + num_to_mask, len(src_ids))):
80             if i in maskable and random.random() < 0.7:
81                 src_ids[i] = self.unk_id
82         return src_ids

```

```

1  def select_vi2zh_window(indices: List[int], epoch: int, ratio: float) -> List[int]:
2      """
3      Chọn một "cửa sổ" con của các mẫu vi2zh cho mỗi epoch
4      sao cho xấp xỉ ratio số mẫu vi2zh được dùng.
5      """
6      if not indices or ratio <= 0:
7          return []
8      total = len(indices)
9      window = max(1, int(math.ceil(total * min(ratio, 1.0))))
10     start = ((epoch - 1) * window) % total
11     end = start + window
12     if end <= total:
13         return indices[start:end]
14     wrap = end - total
15     return indices[start:] + indices[:wrap]
16
17
18     def build_train_loader(
19         train_dataset: BidirectionalTranslationDataset,
20         epoch: int,
21         config: RopeConfig,
22     ) -> Tuple[DataLoader, int]:
23         """
24         Mỗi epoch: lấy toàn bộ mẫu zhvi,
25         cộng thêm một cửa sổ vi2zh quay vòng theo epoch.
26         """
27         active = list(train_dataset.zh2vi_indices)
28         vi_slice = select_vi2zh_window(
29             train_dataset.vi2zh_indices, epoch, config.vi2zh_epoch_ratio

```

```

30     )
31     active.extend(vi_slice)
32
33     sampler = SubsetRandomSampler(active)
34     loader = DataLoader(
35         train_dataset,
36         batch_size=config.batch_size,
37         sampler=sampler,
38         # collate_fn, num_workers,... được thiết lập phù hợp với cấu hình.
39     )
40     return loader, len(vi_slice)

```

`BidirectionalTranslationDataset` xây dựng một tập dữ liệu song ngữ hai chiều, trong đó mỗi mẫu gồm câu nguồn, câu đích và nhãn hướng dịch ("`zh2vi`" hoặc "`vi2zh`"). Trong chế độ huấn luyện, các cặp câu được duyệt luân phiên: chỉ số chẵn gán hướng Hoa→Việt (gắn thêm token "<2vi>" vào đầu câu nguồn), chỉ số lẻ gán hướng Việt→Hoa (gắn "<2zh>"). Trong chế độ validation, mọi mẫu đều được chuẩn hoá về hướng Hoa→Việt để việc đánh giá tập trung vào chiều dịch chính.

Khi truy cập một phần tử, câu nguồn được mã hoá bằng `SentencePiece` (kèm language token) và cắt về `max_len`; hàm `apply_span_masking` với xác suất nhỏ chọn một đoạn 1–2 token thường và thay bằng "<unk>", giúp mô hình học biểu diễn ổn định hơn dưới nhiễu nhẹ ở phía nguồn. Câu đích được bọc BOS/EOS, cắt về `max_len` và cả hai được trả về dưới dạng tensor để đưa vào mô hình.

Hai danh sách chỉ số `zh2vi_indices` và `vi2zh_indices` lưu vị trí các mẫu theo từng hướng dịch. Hàm `select_vi2zh_window` định nghĩa một lịch chọn mẫu cho hướng Việt→Hoa: mỗi epoch chỉ lấy một “cửa sổ” con có kích thước tỉ lệ với `vi2zh_epoch_ratio`, và cửa sổ này được trượt tuần tự qua danh sách chỉ số. Hàm `build_train_loader` xây dựng `DataLoader` huấn luyện bằng cách luôn lấy toàn bộ mẫu Hoa→Việt, sau đó ghép thêm cửa sổ Việt→Hoa tương ứng với epoch hiện tại, giúp ưu tiên học tốt chiều Hoa→Việt nhưng vẫn khai thác đều đặn thông tin nghịch chiều Việt→Hoa trong suốt quá trình huấn luyện.

Bước 6: Xây dựng khối Transformer với RMSNorm, RoPE, GQA và SwiGLU

```

1 class RMSNorm(nn.Module):
2     def __init__(self, d_model: int, eps: float = 1e-8):
3         super().__init__()
4         self.eps = eps
5         self.weight = nn.Parameter(torch.ones(d_model))
6
7     def forward(self, x: torch.Tensor) -> torch.Tensor:
8         rms = torch.sqrt(torch.mean(x ** 2, dim=-1, keepdim=True) + self.eps)
9         return self.weight * x / rms
10

```

```

11
12 class RoPE(nn.Module):
13     def __init__(self, d_model: int, base: float = 10000.0):
14         super().__init__()
15         inv_freq = 1.0 / (base ** (torch.arange(0, d_model, 2).float() / d_model))
16         self.register_buffer("inv_freq", inv_freq)
17         self._cos = None
18         self._sin = None
19         self._seq_len_cached = 0
20
21     def _maybe_update_cache(self, seq_len: int, device, dtype):
22         if seq_len > self._seq_len_cached or self._cos is None or self._cos.device !=
                device:
23             self._seq_len_cached = seq_len
24             positions = torch.arange(seq_len, device=device, dtype=dtype)
25             freqs = torch.outer(positions, self.inv_freq.to(device))
26             self._cos = freqs.cos()
27             self._sin = freqs.sin()
28
29     def forward(self, x: torch.Tensor, seq_len: Optional[int] = None):
30         seq_len = seq_len or x.size(-2)
31         self._maybe_update_cache(seq_len, x.device, x.dtype)
32         return self._cos[:seq_len], self._sin[:seq_len]
33
34
35 def apply_rope(x: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor) -> torch.Tensor:
36     x1 = x[..., 0::2]
37     x2 = x[..., 1::2]
38     cos = cos.unsqueeze(0).unsqueeze(0)
39     sin = sin.unsqueeze(0).unsqueeze(0)
40     rot1 = x1 * cos - x2 * sin
41     rot2 = x1 * sin + x2 * cos
42     return torch.stack([rot1, rot2], dim=-1).flatten(-2)
43
44
45 class FFN_SwiGLU(nn.Module):
46     def __init__(self, d_model: int, d_ff: int, dropout: float):
47         super().__init__()
48         self.d_ff = d_ff
49         self.linear1 = nn.Linear(d_model, 2 * d_ff, bias=False)
50         self.linear2 = nn.Linear(d_ff, d_model, bias=False)
51         self.dropout = nn.Dropout(dropout)
52
53     def forward(self, x: torch.Tensor) -> torch.Tensor:
54         h = self.linear1(x)
55         g, v = h[..., : self.d_ff], h[..., self.d_ff :]
56         s = g * torch.sigmoid(g)
57         out = self.linear2(s * v)
58         return self.dropout(out)
59
60
61 class GroupedQueryAttentionRoPE(nn.Module):
62     def __init__(self, d_model: int, n_heads: int, n_kv_heads: int, dropout: float,
                    rope_base: float):

```



```

63     super().__init__()
64     assert n_heads % n_kv_heads == 0
65     self.n_heads = n_heads
66     self.n_kv_heads = n_kv_heads
67     self.n_groups = n_heads // n_kv_heads
68     self.d_k = d_model // n_heads
69     self.W_q = nn.Linear(d_model, n_heads * self.d_k, bias=False)
70     self.W_k = nn.Linear(d_model, n_kv_heads * self.d_k, bias=False)
71     self.W_v = nn.Linear(d_model, n_kv_heads * self.d_k, bias=False)
72     self.W_o = nn.Linear(d_model, d_model, bias=False)
73     self.dropout = nn.Dropout(dropout)
74     self.scale = math.sqrt(self.d_k)
75     self.rope = RoPE(self.d_k, base=rope_base)
76
77     def forward(self, q, k, v, key_padding_mask=None, attn_mask=None):
78         B, T_q = q.size(0), q.size(1)
79         T_k = k.size(1)
80
81         Q = self.W_q(q).view(B, T_q, self.n_heads, self.d_k).transpose(1, 2)
82         K = self.W_k(k).view(B, T_k, self.n_kv_heads, self.d_k).transpose(1, 2)
83         V = self.W_v(v).view(B, T_k, self.n_kv_heads, self.d_k).transpose(1, 2)
84
85         cos_q, sin_q = self.rope(Q, T_q)
86         cos_k, sin_k = self.rope(K, T_k)
87         Q = apply_rope(Q, cos_q, sin_q)
88         K = apply_rope(K, cos_k, sin_k)
89
90         K = K.repeat_interleave(self.n_groups, dim=1)
91         V = V.repeat_interleave(self.n_groups, dim=1)
92
93         scores = torch.matmul(Q, K.transpose(-2, -1)) / self.scale
94         if key_padding_mask is not None:
95             scores = scores.masked_fill(key_padding_mask.unsqueeze(1).unsqueeze(2),
96                                         float("-inf"))
97         if attn_mask is not None:
98             scores = scores.masked_fill(attn_mask.unsqueeze(0).unsqueeze(0), float("-inf"))
99
100        attn = F.softmax(scores, dim=-1)
101        attn = self.dropout(attn)
102        out = torch.matmul(attn, V)
103        out = out.transpose(1, 2).contiguous().view(B, T_q, -1)
104        return self.W_o(out)
105
106    class EncoderLayer(nn.Module):
107        def __init__(self, config: RopeConfig):
108            super().__init__()
109            self.ln1 = RMSNorm(config.d_model)
110            self.self_attn = GroupedQueryAttentionRoPE(
111                config.d_model, config.n_heads, config.n_kv_heads, config.dropout, config.
112                    rope_base
113            )
114            self.ln2 = RMSNorm(config.d_model)

```

```

114         self.ffn = FFN_SwiGLU(config.d_model, config.d_ff, config.dropout)
115         self.dropout = nn.Dropout(config.dropout)
116
117     def forward(self, x, src_pad_mask=None):
118         x1 = self.ln1(x)
119         attn = self.self_attn(x1, x1, x1, key_padding_mask=src_pad_mask)
120         x = x + self.dropout(attn)
121         x2 = self.ln2(x)
122         return x + self.ffn(x2)
123
124
125 class DecoderLayer(nn.Module):
126     def __init__(self, config: RopeConfig):
127         super().__init__()
128         self.ln1 = RMSNorm(config.d_model)
129         self.self_attn = GroupedQueryAttentionRoPE(
130             config.d_model, config.n_heads, config.n_kv_heads, config.dropout, config.
131                 rope_base
132         )
133         self.ln2 = RMSNorm(config.d_model)
134         self.cross_attn = GroupedQueryAttentionRoPE(
135             config.d_model, config.n_heads, config.n_kv_heads, config.dropout, config.
136                 rope_base
137         )
138         self.ln3 = RMSNorm(config.d_model)
139         self.ffn = FFN_SwiGLU(config.d_model, config.d_ff, config.dropout)
140         self.dropout = nn.Dropout(config.dropout)
141
142     def forward(self, y, enc_out, tgt_pad_mask=None, tgt_causal_mask=None, src_pad_mask
143         =None):
144         y1 = self.ln1(y)
145         self_attn = self.self_attn(
146             y1, y1, y1,
147             key_padding_mask=tgt_pad_mask,
148             attn_mask=tgt_causal_mask,
149         )
150         y = y + self.dropout(self_attn)
151
152         y2 = self.ln2(y)
153         cross_attn = self.cross_attn(y2, enc_out, enc_out, key_padding_mask=
154             src_pad_mask)
155         y = y + self.dropout(cross_attn)
156
157         y3 = self.ln3(y)
158         return y + self.ffn(y3)
159
160
161 class TransformerModel(nn.Module):
162     def __init__(self, config: RopeConfig, vocab_size: int):
163         super().__init__()
164         self.config = config
165         self.embedding = nn.Embedding(vocab_size, config.d_model, padding_idx=0)
166         self.emb_dropout = nn.Dropout(config.dropout)

```

```

64     self.encoder_layers = nn.ModuleList(
65         [EncoderLayer(config) for _ in range(config.num_encoder_layers)]
66     )
67     self.encoder_final_ln = RMSNorm(config.d_model)
68
69     self.decoder_layers = nn.ModuleList(
70         [DecoderLayer(config) for _ in range(config.num_decoder_layers)]
71     )
72     self.decoder_final_ln = RMSNorm(config.d_model)
73
74     self.output_bias = nn.Parameter(torch.zeros(vocab_size))
75     self.emb_scale = math.sqrt(config.d_model)
76     self._init_weights()
77
78     def _init_weights(self):
79         for p in self.parameters():
80             if p.dim() > 1:
81                 nn.init.xavier_uniform_(p)
82
83     def forward(self, src_ids: torch.Tensor, tgt_ids: torch.Tensor) -> torch.Tensor:
84         src_pad = (src_ids == 0)
85         tgt_pad = (tgt_ids == 0)
86
87         tgt_in = tgt_ids[:, :-1]
88         tgt_pad_in = tgt_pad[:, :-1]
89         T = tgt_in.size(1)
90         tgt_causal = torch.triu(
91             torch.ones(T, T, dtype=torch.bool, device=src_ids.device),
92             diagonal=1,
93         )
94
95         src_emb = self.emb_dropout(self.embedding(src_ids) * self.emb_scale)
96         enc_out = src_emb
97         for layer in self.encoder_layers:
98             enc_out = layer(enc_out, src_pad)
99         enc_out = self.encoder_final_ln(enc_out)
100
101         tgt_emb = self.emb_dropout(self.embedding(tgt_in) * self.emb_scale)
102         dec_out = tgt_emb
103         for layer in self.decoder_layers:
104             dec_out = layer(dec_out, enc_out, tgt_pad_in, tgt_causal, src_pad)
105         dec_out = self.decoder_final_ln(dec_out)
106
107         return F.linear(dec_out, self.embedding.weight, self.output_bias)

```

Đoạn mã trên hiện thực toàn bộ các khối kiến trúc cốt lõi của mô hình dịch Hoa-Việt. Lớp `RMSNorm` chuẩn hoá độ lớn vector theo chuẩn căn-trung-bình-bình-phương, giúp ổn định gradient khi số lớp encoder/decoder lớn. Khối `RoPE` và hàm `apply_rope` mã hoá vị trí bằng phép quay trên từng cặp chiều của query/key, nhúng thông tin vị trí tương đối trực tiếp vào không gian attention.

Lớp `FFN_SwiGLU` hiện thực tầng feed-forward với cơ chế SwiGLU: đầu tiên chiếu biểu diễn sang

không gian ẩn gấp đôi, sau đó tách thành hai nhánh *gate* và *value*, *gate* được kích hoạt bằng SiLU và dùng để điều biến nhánh *value* trước khi chiếu ngược về **d_model**. Cách thiết kế này cho phép tầng FFN biểu đạt các quan hệ phi tuyến sâu hơn so với FFN dùng ReLU/GELU.

Lớp **GroupedQueryAttentionRoPE** kết hợp ba ý tưởng: multi-head attention, grouped-query và RoPE. Ở đây, mỗi head query vẫn độc lập, nhưng các head được chia thành các nhóm cùng chia sẻ một cặp key-value, giúp giảm chi phí KV cache trong khi vẫn giữ được nhiều hướng chú ý khác nhau. RoPE được áp dụng lên Q và K trước khi tính attention scores, đảm bảo mô hình tận dụng tốt thông tin thứ tự.

EncoderLayer và **DecoderLayer** sắp xếp các khối trên theo sơ đồ pre-norm: mỗi lớp encoder gồm RMSNorm → grouped-query self-attention with RoPE → residual → RMSNorm → FFN-SwiGLU → residual; mỗi lớp decoder gồm RMSNorm → masked grouped-query self-attention with causal mask → residual, tiếp theo là RMSNorm → grouped-query cross-attention lên encoder outputs → residual, và cuối cùng RMSNorm → FFN-SwiGLU → residual.

Lớp **TransformerModel** gán kết toàn bộ: embedding chung cho cả nguồn và đích, một stack nhiều **EncoderLayer** để tạo contextual representations cho câu nguồn, một stack nhiều **DecoderLayer** để sinh câu đích theo cơ chế autoregressive với causal mask, và một tầng tuyến tính cuối cùng sử dụng lại trọng số embedding để sinh logits trên không gian từ vựng. Đây là hiện thực cụ thể của kiến trúc Transformer Encoder-Decoder được mô tả trong phần lý thuyết cho bài toán dịch Hoa-Việt.

Bước 7: Hàm mất mát với label smoothing

```

1 class LabelSmoothedCrossEntropyLoss(nn.Module):
2     def __init__(self, smoothing: float = 0.1, ignore_index: int = 0):
3         super().__init__()
4         self.smoothing = smoothing
5         self.ignore_index = ignore_index
6
7     def forward(self, logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
8         vocab_size = logits.size(-1)
9         log_probs = F.log_softmax(logits, dim=-1)
10
11         # Không tính loss trên vị trí padding
12         mask = targets != self.ignore_index
13
14         with torch.no_grad():
15             # Phân phối "mềm" thay vì one-hot
16             true_dist = torch.full_like(
17                 log_probs, self.smoothing / (vocab_size - 1)
18             )
19             true_dist.scatter_(1, targets.unsqueeze(1), 1.0 - self.smoothing)
20             true_dist[targets == self.ignore_index] = 0.0
21
22         loss = -(true_dist * log_probs).sum(dim=-1)
23         loss = loss.masked_fill(~mask, 0.0)
24         return loss.sum() / mask.sum().clamp(min=1)

```

Hàm mất mát `LabelSmoothedCrossEntropyLoss` thay thế phân phối one-hot truyền thống bằng một phân phối *mềm*: thay vì gán xác suất 1.0 cho token đích duy nhất và 0.0 cho tất cả token còn lại, một phần nhỏ xác suất (*smoothing*) được phân bổ đều lên các lớp khác nhau trong vocab. Điều này làm giảm độ tự tin cực đoan của mô hình, đóng vai trò như một dạng regularization giúp mô hình bớt overfit và ổn định hơn trên dữ liệu nhiễu.

Trong quá trình tính loss, các logits được đưa qua `log_softmax`, sau đó nhân với phân phối mục tiêu đã được làm mượt `true_dist` để thu được giá trị cross-entropy tại từng vị trí. Tham số `ignore_index` cho phép loại bỏ hoàn toàn các vị trí padding (thường tương ứng với token `<pad>`), bằng cách đặt phân phối mục tiêu về 0 tại đó và không tính các phần tử này vào mẫu số khi chuẩn hoá loss trung bình. Cách hiện thực này đảm bảo hàm mất mát vừa phản ánh tốt chất lượng dự đoán trên token thật, vừa tránh việc padding làm sai lệch giá trị loss trong quá trình huấn luyện.

```
Epoch 39/40
vi→zh coverage: 10598/15140 (~70.0% per epoch)
Train Loss: 0.1666
Valid Loss: 0.8484
Valid BLEU: 63.30
Learning Rate: 0.000045
Epoch 40: 100%|██████████| 101/101 [00:28<00:00, 3.49it/s, loss=0.1691, lr=0.000044]
Epoch 40: 100%|██████████| 101/101 [00:28<00:00, 3.49it/s, loss=0.1691, lr=0.000044]

Epoch 40/40
vi→zh coverage: 10598/15140 (~70.0% per epoch)
Train Loss: 0.1665
Valid Loss: 0.8474
Valid BLEU: 63.35
Learning Rate: 0.000044
✓ Saved checkpoint to ./checkpoints_bidirectional/checkpoint_epoch_40.pt
```

Hình 12: Kết quả các epoch cuối sau khi pre-training.

Bước 8: Thành phần contrastive và mất mát song ngữ

```
1 import torch.nn as nn
2 import torch.nn.functional as F
3
4 # Projection head: ánh xạ vector ẩn sang không gian embedding đối lập
5 class ProjectionHead(nn.Module):
6     def __init__(self, input_dim: int, proj_dim: int):
7         super().__init__()
8         self.net = nn.Sequential(
9             nn.Linear(input_dim, input_dim),
10            nn.ReLU(),
11            nn.Linear(input_dim, proj_dim),
12        )
13
14    def forward(self, x):
15        # Chuẩn hoá L2 để dùng cosine similarity
```

```

16         return F.normalize(self.net(x), dim=-1)
17
18
19 def encode_context(model, src_ids, pad_id):
20     """Chạy encoder để lấy biểu diễn ngữ cảnh cho toàn bộ câu."""
21     src_pad_mask = (src_ids == pad_id)
22     src_emb = model.embedding(src_ids) * model.emb_scale
23     src_input = model.emb_dropout(src_emb)
24
25     enc_out = src_input
26     for layer in model.encoder_layers:
27         enc_out = layer(enc_out, src_pad_mask)
28     enc_out = model.encoder_final_ln(enc_out)
29     return enc_out
30
31
32 def mean_pool(enc_out, ids, pad_id, special_ids):
33     """Mean pooling trên các token nội dung, bỏ pad + special tokens."""
34     mask = (ids != pad_id)
35     for special in special_ids:
36         mask = mask & (ids != special)
37     mask = mask.float()
38
39     summed = (enc_out * mask.unsqueeze(-1)).sum(dim=1)
40     denom = mask.sum(dim=1, keepdim=True).clamp(min=1.0)
41     return summed / denom
42
43
44 def contrastive_loss(z_a, z_b, tau):
45     """
46     InfoNCE loss hai chiều giữa hai view (z_a, z_b):
47     mỗi câu dùng bản dịch của chính nó là positive, câu khác là negative.
48     """
49     sim = torch.matmul(z_a, z_b.transpose(0, 1)) / tau
50     labels = torch.arange(sim.size(0), device=sim.device)
51
52     loss_i = F.cross_entropy(sim, labels)
53     loss_j = F.cross_entropy(sim.transpose(0, 1), labels)
54     return 0.5 * (loss_i + loss_j)
55
56
57 def compute_crosslingual_loss(
58     model,
59     projection,
60     ids_zh_vi,
61     ids_vi_vi,
62     ids_vi_zh,
63     ids_zh_zh,
64     cl_config,
65     pad_id,
66     special_ids,
67 ):
68     if ids_zh_vi.size(0) < 2:
69         return torch.zeros(1, device=config.device)

```

```

70
71 # 1) Mã hoá 4 view của cùng cặp câu
72 enc_zh_vi = encode_context(model, ids_zh_vi, pad_id)
73 enc_vi_vi = encode_context(model, ids_vi_vi, pad_id)
74 enc_vi_zh = encode_context(model, ids_vi_zh, pad_id)
75 enc_zh_zh = encode_context(model, ids_zh_zh, pad_id)
76
77 # 2) Mean pooling + projection sang không gian đối lập
78 z_zh_vi = projection(mean_pool(enc_zh_vi, ids_zh_vi, pad_id, special_ids))
79 z_vi_vi = projection(mean_pool(enc_vi_vi, ids_vi_vi, pad_id, special_ids))
80 z_vi_zh = projection(mean_pool(enc_vi_zh, ids_vi_zh, pad_id, special_ids))
81 z_zh_zh = projection(mean_pool(enc_zh_zh, ids_zh_zh, pad_id, special_ids))
82
83 # 3) Hai loss đối lập song ngữ: "vi-space" và "zh-space"
84 cl_vi_space = contrastive_loss(
85     z_zh_vi, z_vi_vi, cl_config.contrastive_tau
86 )
87 cl_zh_space = contrastive_loss(
88     z_vi_zh, z_zh_zh, cl_config.contrastive_tau
89 )
90
91 # Trung bình hai không gian
92 return 0.5 * (cl_vi_space + cl_zh_space)

```

Khối `ProjectionHead` đóng vai trò ánh xạ các vector ẩn từ encoder (kích thước `d_model`) sang một không gian embedding đối lập có chiều thấp hơn (`proj_dim`), kèm chuẩn hoá L2 để đo tương tự giữa hai câu tương ứng trở thành cosine similarity. Mỗi câu sau khi đi qua encoder được *mean pooling* trên các token nội dung (loại bỏ `pad`, `BOS/EOS` và language tokens), rồi đưa qua projection head để thu được một vector biểu diễn duy nhất cho toàn bộ câu.

Hàm `contrastive_loss` hiện thực một biến thể InfoNCE hai chiều: ma trận `sim` chứa cosine similarity (sau khi chia cho nhiệt độ `tau`) giữa từng cặp câu trong batch; mỗi câu sử dụng bản dịch của chính nó làm *positive*, các câu còn lại trong batch làm *negative*. Loss được tính theo cả hai hướng (dùng `z_a` truy vấn `z_b` và ngược lại), sau đó lấy trung bình để cân bằng hai phía.

Hàm `compute_crosslingual_loss` tạo ra bốn view cho mỗi cặp song ngữ: Hoa với token "<2vi>" và "<2zh>", Việt với token "<2vi>" và "<2zh>". Các view này lần lượt được mã hoá, pooling và đưa qua projection head, từ đó xây dựng hai không gian đối lập: một "vi-space" (căn chỉnh `zh_vi` với `vi_vi`), một "zh-space" (căn chỉnh `vi_zh` với `zh_zh`). Việc tối ưu trung bình hai InfoNCE loss trong hai không gian này buộc mô hình đưa các câu dịch tương ứng trong Hoa-Việt tiến gần nhau trong không gian ngữ nghĩa chung, tăng chất lượng biểu diễn cross-lingual ngay tại tầng encoder và hỗ trợ tốt hơn cho nhiệm vụ dịch chính.

```

Contrastive Epoch 19/20
CE Loss: 0.1499
CL Loss: 0.0111
Total Loss: 0.1510
Learning Rate: 0.000010
Contrastive Epoch 20: 100% [██████████] 250/250 [01:32<00:00, 2.71it/s, ce=0.1506, cl=0.0000, λ=0.100, lr=0.000010]

Contrastive Epoch 20/20
CE Loss: 0.1497
CL Loss: 0.0114
Total Loss: 0.1508
Learning Rate: 0.000010

```

Hình 13: Kết quả các epoch cuối sau khi contrastive training.

Bước 9: Kết quả xếp hạng của model được xây dựng

#	Thí sinh	Ngày	ID	Total Score
	Không phải TQKhang	2025-11-19 14:59	6067	68.8
	anhht	2025-11-27 17:02	6965	68.55
	tvan	2025-11-22 03:35	6471	66.95
	Hao Bui Cong	2025-11-20 14:33	6234	65.9
	quanghien	2025-11-28 11:50	7019	65.71

Hình 14: Kết quả xếp hạng của model trong cuộc thi.

Mô hình Transformer Encoder–Decoder đề xuất, với các cải tiến như RMSNorm, RoPE, grouped-query attention, FFN dùng SwiGLU, huấn luyện hai chiều Hoa–Việt kết hợp *label smoothing* và lịch học *warmup + inverse square root*, đã cho kết quả khả quan trên hệ thống chấm điểm trực tuyến của Ban tổ chức. Sau khi huấn luyện xong, mô hình được dùng để sinh dự đoán cho hai file `public_test.csv` và `private_test.csv` theo đúng định dạng yêu cầu, nộp lên hệ thống và thu được điểm tổng **65.71**, xếp hạng **Top 5** như thể hiện trên bảng xếp hạng. Kết quả này cho thấy thiết kế kiến trúc và quy trình tiền xử lý–huấn luyện là hợp lý, đủ sức cạnh tranh với các lời giải hàng đầu trong cuộc thi dịch máy Hoa–Việt.

VI. Hướng phát triển và cải tiến mô hình

Dựa trên nền tảng mô hình *Transformer Encoder–Decoder* với RMSNorm, RoPE, Grouped-Query Attention, FFN-SwiGLU và thành phần contrastive song ngữ đã trình bày, mô hình dịch Hoa–Việt hoàn toàn có thể được phát triển thêm theo nhiều hướng khác nhau. Phần này đề xuất một số hướng mở rộng thực nghiệm nhằm cải thiện hơn nữa chỉ số SacreBLEU và chất lượng dịch thực tế.

VI.1. Thử nghiệm đa cấu hình: kiến trúc, từ vựng và lịch huấn luyện

Một hướng phát triển tự nhiên là khảo sát có hệ thống các siêu tham số đã được cố định trong cấu hình `RopeConfig`, bao gồm kích thước từ vựng, kích thước mô hình và lịch huấn luyện:

- **Kích thước từ vựng (VOCAB_SIZE):** Thử nghiệm các giá trị khác nhau (ví dụ 4k, 8k, 12k, 16k) để đánh đổi giữa: (i) Độ mịn biểu diễn subword (giảm <unk> và cải thiện xử lý từ hiếm) và (ii) Độ khó khi tối ưu trên không gian từ vựng quá lớn.
- **Kích thước mô hình (d_model, d_ff, số lớp encoder/decoder):** So sánh các cấu hình “base” và “large”:
 - Tăng `d_model` và `d_ff` giúp mô hình có năng lực biểu diễn mạnh hơn.
 - Tăng số lớp encoder/decoder giúp nắm bắt ngữ cảnh sâu hơn, nhưng cần thêm regularization (như label smoothing, R-Drop) để tránh overfitting.
- **Số epoch và lịch học:**
 - Huấn luyện lâu hơn với `num_epochs` lớn hơn, kết hợp **early stopping** trên tập validation.
 - Điều chỉnh `warmup_steps` và dạng scheduler (Noam, cosine decay) để mô hình hội tụ ổn định.
 - Thử thay đổi `vi2zh_epoch_ratio` để đánh giá mức độ đóng góp của chiều Việt→Hoa vào việc cải thiện chiều Hoa→Việt.

Các thí nghiệm này có thể được tổ chức theo dạng *grid search* hoặc *random search*, sau đó so sánh SacreBLEU trên tập kiểm tra công khai để lựa chọn cấu hình tối ưu.

VI.2. Khai thác Back-Translation để mở rộng dữ liệu

Bên cạnh bộ song ngữ gốc, một hướng rất hiệu quả là áp dụng *Back-Translation* để tận dụng dữ liệu đơn ngữ tiếng Việt:

- Sử dụng mô hình hiện tại (hoặc một mô hình đơn giản hơn) dịch các câu tiếng Việt đơn ngữ sang tiếng Hoa, tạo thành cặp (`zh_synthetic`, `vi_real`).
- Trộn các cặp câu sinh ra này với bộ song ngữ gốc trong giai đoạn huấn luyện tiếp theo (fine-tuning).

Điểm mạnh của Back-Translation:

- Tận dụng được nguồn dữ liệu đơn ngữ tiếng Việt ngoài tập huấn luyện ban đầu.
- Tăng độ đa dạng về cấu trúc câu, ngữ cảnh và cách diễn đạt ở phía đích (Việt), giúp mô hình học cách sinh câu tự nhiên hơn.
- Có thể kết hợp với chiến lược huấn luyện hai giai đoạn: pre-training với dữ liệu song ngữ gốc, sau đó fine-tuning với tập mở rộng có thêm dữ liệu Back-Translation.

VI.3. Regularization nâng cao: R-Drop và tối ưu hoá SAM

R-Drop Regularization: Trên nền `LabelSmoothedCrossEntropyLoss`, có thể bổ sung R-Drop để giảm phương sai giữa các lần forward pass:

- Với mỗi batch, chạy hai lần forward với cùng đầu vào nhưng khác `dropout mask`.
- Tính thêm một thành phần KL-divergence giữa hai phân phối đầu ra logits:

$$\mathcal{L}_{\text{R-Drop}} = \text{KL}(p_{\theta}^{(1)} \| p_{\theta}^{(2)}) + \text{KL}(p_{\theta}^{(2)} \| p_{\theta}^{(1)}),$$

và cộng với loss dịch chính.

- Cách này khuyến khích mô hình sinh ra phân phối dự đoán nhất quán hơn, giảm overfitting và thường cải thiện BLEU.

SAM (Sharpness-Aware Minimization): Ngoài các optimizer chuẩn như Adam/AdamW, có thể thử *SAM optimizer*:

- SAM không chỉ tối ưu giá trị loss tại một điểm tham số, mà còn cố gắng tìm các vùng lân cận có loss nhỏ (vùng phẳng).
- Trực giác: nghiệm nằm ở “vùng phẳng” thường có khả năng tổng quát tốt hơn, ít nhạy với nhiễu dữ liệu.
- Trong thực nghiệm, SAM cho thấy hiệu quả trên nhiều bài toán thị giác và ngôn ngữ, nên việc áp dụng cho bài toán dịch Hoa–Việt là hướng đáng khảo sát.

VI.4. Mở rộng khung contrastive với cặp dương từ nhiều phiên bản của cùng một câu

Phần trước đã xây dựng một loss contrastive song ngữ dựa trên các view khác nhau của cùng một cặp Hoa-Việt (zh_vi , vi_vi , vi_zh , zh_zh). Một hướng mở rộng là bổ sung một nhánh contrastive theo tinh thần *self-supervised* đơn ngữ, trong đó cặp dương được xây dựng từ các phiên bản khác nhau của chính một câu:

- Tạo nhiều phiên bản (views) của cùng một câu bằng các phép biến đổi:
 - Span masking mạnh hơn ở phía encoder.
 - Dropout ở tầng embedding hoặc encoder.
 - Round-trip translation đơn giản: $x \rightarrow \hat{y} \rightarrow \tilde{x}$ (với $\hat{y}$ là bản dịch tạm), sau đó dùng x và $\tilde{x}$ làm hai views.
- Đặt các phiên bản này làm cặp dương trong InfoNCE:

$$\mathcal{L}_{\text{self-CL}} = \text{InfoNCE}\left(z(x^{(1)}), z(x^{(2)})\right),$$

với negative là các câu khác trong batch.

- Kết hợp $\mathcal{L}_{\text{self-CL}}$ với loss contrastive song ngữ hiện tại và loss dịch:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MT}} + \lambda_{\text{CL-bilingual}} \mathcal{L}_{\text{CL-bilingual}} + \lambda_{\text{CL-self}} \mathcal{L}_{\text{self-CL}}.$$

Cách làm này giúp encoder học được biểu diễn ổn định hơn cho từng câu riêng lẻ, đồng thời hỗ trợ cho việc căn chỉnh song ngữ trong không gian embedding chung.

VI.5. Khảo sát các chiến lược giải mã: beam search, top- k , top- p

Ở giai đoạn suy luận, ngoài beam search cổ điển, có thể khảo sát thêm các chiến lược giải mã khác nhau để đánh đổi giữa độ chính xác và độ tự nhiên của bản dịch:

- **Beam search với điều chỉnh penalty:**
 - Thử các giá trị `beam_size` khác nhau (ví dụ 4, 6, 8).
 - Điều chỉnh *length penalty* để tránh hiện tượng sinh câu quá ngắn hoặc quá dài.
- **Sampling-based decoding:**
 - *Top- k sampling*: tại mỗi bước, chỉ giữ lại k token có xác suất cao nhất và sample trong tập này, phù hợp khi muốn sinh ra nhiều bản dịch đa dạng.
 - *Top- p (nucleus) sampling*: chọn tập nhỏ nhất các token sao cho tổng xác suất $\geq p$ (ví dụ $p = 0,9$), sau đó sampling trong tập đó.

- **Kết hợp search và reranking:**

- Sinh ra nhiều candidate bằng beam search hoặc sampling.
- Dùng một language model tiếng Việt (huấn luyện riêng từ tập `vi_lines`) để rerank theo độ trôi chảy, chọn bản dịch tự nhiên và phù hợp ngữ pháp hơn.

Các chiến lược này không thay đổi trọng số mô hình, nhưng có thể tác động đáng kể đến chất lượng dịch cuối cùng, đặc biệt về độ mượt và tự nhiên của câu tiếng Việt.

VI.6. Vòng lặp suy luận, phân tích lỗi và cải thiện mô hình

Một bước quan trọng để phát triển mô hình bền vững là xây dựng vòng lặp phản hồi dựa trên kết quả suy luận:

1. Suy luận trên tập kiểm tra/validation:

- Chạy inference trên `public_test.zh` và một phần `train.zh` được dành riêng làm validation nội bộ.
- Lưu lại đồng thời bản dịch của mô hình và bản dịch tham chiếu (khi có).

2. Lọc và phân loại các dự đoán sai:

- Ưu tiên phân tích các câu có BLEU câu-level thấp hoặc chênh lệch lớn về độ dài so với tham chiếu.
- Gắn nhãn thủ công một số nhóm lỗi tiêu biểu:
 - Dịch sai thuật ngữ chuyên ngành.
 - Sai trật tự cú pháp (chủ-vị-tân, vị trí trạng ngữ).
 - Bỏ sót hoặc lặp cụm từ.
 - Xử lý chưa tốt tên riêng, số, đơn vị đo lường.

3. Rút ra hướng điều chỉnh mô hình:

- Nếu xuất hiện nhiều lỗi về thuật ngữ: bổ sung từ điển song ngữ hoặc lớp hậu xử lý (post-processing) cho các trường hợp đặc biệt.
- Nếu bản dịch hay lặp từ/cụm: điều chỉnh chiến lược decoding (thêm penalty cho *repetition*) hoặc tăng regularization (R-Drop, SAM).
- Nếu lỗi chủ yếu liên quan đến trật tự cú pháp: tăng số lớp encoder/decoder hoặc bổ sung dữ liệu Back-Translation với cấu trúc câu đa dạng hơn.

4. Huấn luyện lại hoặc fine-tuning với thông tin mới:

- Tạo tập “hard examples” gồm những câu mô hình dịch sai nhiều để fine-tuning tập trung.
- Lặp lại bước đo SacreBLEU và phân tích lỗi để đánh giá hiệu quả của từng thay đổi.

Thông qua vòng lặp này, mô hình không chỉ được cải thiện về các chỉ số tổng quan như SacreBLEU, mà còn được tinh chỉnh dần theo những dạng lỗi thực tế thường gặp trong bài toán dịch Hoa–Việt, phù hợp với yêu cầu của đề thi và các ứng dụng triển khai trong môi trường thật.

Phụ lục

1. **Code:** Các file code được đề cập trong bài có thể được tải tại [tại đây](#).
2. **Dataset:** Bộ dữ liệu có thể được tải tại [tại đây](#).
3. **Contest:** Thể lệ về cuộc thi có thể đọc tại [tại đây](#).
4. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

AIO_QAs-Verified

🔒 Private group · 1.4K members



Hình 15: Hình ảnh group facebook AIO Q&A